

UK Train Rides

Capstone Project Documentation

Team Innovators — DEPI CAPSTONE PROJECT

Project Team

Team leader	Mohamed Elsayed
Team member 2	Abdulrahman Abdulrazek
Team member 3	Gaber Tahoon
Team member 4	Ahmed Fawzy

Documentation Contents

1. Business Understanding Report (BUR)
2. Data Cleaning Report
3. Exploratory Data Analysis (EDA) Report
4. Power BI Dashboard Documentation
5. Forecasting: Railway Journey Predictor

Table of Contents

Business Understanding Report

Team Innovators - DEPI CAPSTONE PROJECT - UK Train Rides

Project Section 1 of 5: Business Understanding Report (BUR)

1. Business Problem Statement

PS 01:

The company wants to analyze service delays to identify the main reasons customers request ticket refunds in order to reduce financial losses caused by delays.

PS 02:

The company wants to analyse ticket pricing and purchase patterns to evaluate the impact of the 50% advance-purchase discount on revenue, since a large percentage of customers buy tickets in advance, and the high discount may reduce overall profitability.

PS 03:

The company wants to analyse ticket purchases at stations and purchase times to identify peak station demand periods in order to reduce the crowding, and evaluate the ticket price.

2. Business Processes

A customer selects a route (depart → arrive station) and ticket type (Advance - First Class - Anytime) and completes the purchase either online or at the station.

The system records the transaction and schedules the journey for a specific date. On the (Date of Journey), the actual arrival time is compared against the scheduled time.

If a disruption occurs (delay - canceled), the customer may then initiate a "Refund Request" through the company's customer service process.

3. Stakeholders

Stakeholder	Interest	Type
Sales Management	Accurate pricing strategies to maximize revenue.	Internal
Operation Management	Reducing operational pressure and optimizing workflow efficiency.	Internal
Marketing Team	Fine-tuning discount structures and promotional campaigns.	Internal
Customer Service	Reducing refund requests and improving satisfaction for unhappy customers.	Internal
Government	Compliance with regulations, taxation, and legal frameworks.	External
Passengers	Reliable service, fair pricing, and a smooth user experience.	External

4. Analytical Questions & KPIs

Sales Management

KPI

- Gross Sales
- Net. Revenue (Revenue after refunds)
- Avg. Ticket Price
- Discount Rate %
- Price Realization % (Price / Price before discount)
- Total Loss from Refund (Refund Amount)
- Total Loss from Discount (Discount Leak)

Questions

- What is the highest-revenue Purchase Type, and does it differ in average ticket value?
- Revenue mix by Ticket Type (Advance/Off-Peak/Anytime)
- **What % of total revenue comes from the top 5 lines?** (Concentration risk)
- **Which Railcard × Ticket Type combination drives the highest discount leakage relative to its revenue share?**
- Revenue per ticket Classes
- Refund Impact on Revenue (Trend)
- Revenue over time
- Top 5 high-revenue lines

Operations

KPI (per day)

- Refund Request Rate %
- Delay Rate %
- On Time Rate %
- Cancellation Rate %
- Peak Hour
- Total Journeys
- Avg. Delay Mins

Questions

- **What are the reasons for the delay** that make customers **request a refund**?
- **At what times do the reasons for delay** affect the **journey delay**?
- Top 5 high-demand lines
- What is the busiest (time of day - day of the week) (peak time)
- For each reason for delay, Avg. Delay Time.
- What is the expected journey time for each line?
- What is the estimated time between ticket purchase and journey time per Ticket Type?
- What is the most common purchase type?
- What are Departure Time Peak Hours Range?

Passengers & Marketing

KPI

- Total Passengers
- Total Passengers have a Railcard
- Special Need Percentage
- Avg. Ticket Price

Questions

- What is the Distribution of person who holds a Railcard (Adult - Senior - Disabled) (Pie Chart)
- Purchase Method (Contactless - Debit)
- Channel (Sales - Revenue - Volume) rate between online and station.
- **Which Ticket Type/Railcard combination is most associated with each Purchase Type?**
- **Is there a relationship between Payment Method and Refund Request rate?**

5. Data Sources

Data Source	Description	Columns
Railway Transactions	Individual train ticket purchase records	Date of Purchase, Time of Purchase, Purchase Type, Price, Refund Request
Stations	Departure and arrival station information	Departure Station, Arrival Destination
Journey	Scheduled and actual journey timing details	Date of Journey, Departure Time, Arrival Time, Actual Arrival Time
Ticket	Ticket class, type, and pricing information	Ticket Class, Ticket Type
Passenger	Railcard and payment method details	Railcard, Payment Method
Delay	Journey status and reasons for delay	Reason for Delay, Journey Status

6. Data Dictionary

Field	Description	Data Type	Example
Transaction ID	Unique identifier for an individual train ticket purchase	CHAR(23)	da8a6ba8-b3dc-4677-b176
Date of Purchase	Date the ticket was purchased	Date (YYYY-MM-DD)	2023-12-08
Time of Purchase	Time the ticket was purchased	Time (HH:MM:SS)	12:41:11
Purchase Type	Whether the ticket was purchased online or directly at a train station	VARCHAR(255)	Online, Station
Payment Method	Payment method used to purchase the ticket (Contactless, Credit Card, or Debit Card)	VARCHAR(255)	Contactless, Credit Card, Debit Card

Field	Description	Data Type	Example
Railcard	Whether the passenger is a National Railcard holder (Adult, Senior, or Disabled) or not (None). Railcard holders get 1/3 off their ticket purchases.	VARCHAR(255)	Adult, Senior, Disabled, None
Ticket Class	Seat class for the ticket (Standard or First)	VARCHAR(255)	Standard, First Class
Ticket Type	When you bought or can use the ticket. Advance tickets are 1/2 off and must be purchased at least a day prior to departure. Off-Peak tickets are 1/4 off and must be used outside of peak hours (weekdays between 6-8am and 4-6pm). Anytime tickets are full price and can be bought and used at any time during the day.	VARCHAR(255)	Advance, Anytime, Off-Peak
Price	Final cost of the ticket	Integer	43
Departure Station	Station to board the train	VARCHAR(255)	London Paddington
Arrival Destination	Station to exit the train	VARCHAR(255)	Liverpool Lime Street
Date of Journey	Date the train departed	Date (YYYY-MM-DD)	2024-01-01
Departure Time	Time the train departed	Time (HH:MM:SS)	11:00:00
Arrival Time	Time the train was scheduled to arrive at its destination (can be on the day after departure)	Time (HH:MM:SS)	13:30:00
Actual Arrival Time	Time the train arrived at its destination (can be on the day after departure)	Time (HH:MM:SS)	13:30:00
Journey Status	Whether the train was on time, delayed, or cancelled	VARCHAR(255)	On Time, Delayed, Cancelled
Reason for Delay	Reason for the delay or cancellation	VARCHAR(255)	Signal Failure, Weather Conditions, Staffing
Refund Request	Whether the passenger requested a refund after a delay or cancellation	Boolean (Yes/No)	Yes, No

7. Assumptions and Constraints

Data Cleaning Transformations

- **Railcard** — Missing **Railcard** values in raw data (no railcard held) were recoded as "Null" (example refers to the raw/pre-cleaning state).
- **Reason for Delay** — Missing **Reason for Delay** values occur only for "On Time" journeys (logically expected) and were filled as "No Delay".

- **Reason for Delay** — Inconsistent raw labels in **Reason for Delay** were standardized to one canonical label each (e.g., "Signal failure"/"Signal Failure" → "Signal Failure"; "Staffing" → "Staff Shortage"; "Weather" → "Weather Conditions").
- **Actual Arrival Time** is missing by definition for all Cancelled journeys (the train never ran) — not treated as a data quality issue, and excluded from delay-time calculations for that status.

Known Data Quality Issues

- **Arrival Time / Actual Arrival Time** can fall on the day after departure (overnight services). Journey duration and delay calculations must account for day-rollover; otherwise ~3% of journeys (overnight routes) produce invalid negative durations.
- "Edinburgh" and "Edinburgh Waverley" appear as two distinct values in **Arrival Destination** for what is the same physical station — Route-level aggregations (Top N lines, revenue per line) will undercount this destination unless merged.

Business Logic Assumptions

- **Refund Request = Yes** is treated as a successful/full refund of **Price** — the source data has no separate "Refund Approved/Rejected" status, so "Total Refunded Amount" assumes every request was honored in full.
- **Price** reflects the amount actually charged (post-discount). **Price before discount** and **Total Discount %** are reverse-derived by assuming Ticket Type discount and Railcard discount combine **multiplicatively** (e.g., Advance + Railcard = $1 - (0.5 \times 0.667) \approx 66.7\%$ off), since the business has not explicitly confirmed whether discounts stack multiplicatively or additively.

Data Cleaning Report

Team Innovators - DEPI CAPSTONE PROJECT - UK Train Rides

Project Section 2 of 5: Data Cleaning

Source notebook: `Data_Cleaning.ipynb` | Input: `railway.csv` | Output: `cleaned_railway.csv`

0. Purpose of This Document

This document explains the data cleaning phase of the project: every issue found in the raw `railway.csv` dataset, why each issue mattered for the business questions defined in the Business Understanding Report, how it was investigated and fixed in code, and what the resulting change was in the data. It also documents every new column engineered during this phase, since feature engineering and cleaning were performed together in the same notebook. Nothing described here is a summary — it reflects every cleaning and transformation step actually executed in `Data_Cleaning.ipynb`, in the order it was executed.

1. Dataset Overview: Before and After

The raw dataset (`railway.csv`) contains 31,653 rows and 18 columns, all read in as text/object dtype by pandas except Price, which pandas inferred as an integer. The cleaned dataset (`cleaned_railway.csv`) contains the same 31,653 rows (no rows were dropped or added during cleaning) but 27 columns — the original 18 plus 9 new engineered columns.

Aspect	Before (raw)	After (cleaned)	Note
Rows	31,653	31,653	No rows added or removed — cleaning was non-destructive at the row level.
Columns	18	27	9 new columns engineered (see Section 4).
Railcard missing values	20,918 (NaN)	0 — recoded to "No Railcard"	Missingness was meaningful (no railcard held), not a data error.
Reason for Delay distinct labels	9 raw labels (incl. duplicates/casing + NaN)	6 canonical labels, 0 missing	Standardized casing + filled logical NaNs with "No Delay".
Arrival Destination distinct values	32	31	"Edinburgh" merged into "Edinburgh Waverley".
Date / Time columns	Stored as text (str)	Parsed as datetime/timedelta, then re-exported as HH:MM:SS strings	Enabled date/time arithmetic during cleaning; final CSV kept human-readable.
Journey Duration negative values	N/A (column did not exist)	0 (after +1440 min correction)	Overnight day-rollover handled explicitly.

2. Step 1 — Identify & Explore Issues

Before changing anything, the raw data was loaded and inspected using `df.head()`, `df.info()`, and targeted column checks (`.isna().sum()`, `.unique()`) to build a full list of issues. Six issues were logged directly in the notebook, plus one additional business-rule check that was investigated but required no fix. Each is documented below.

Issue 01 — Missing Railcard values

Why: A missing value in Railcard is ambiguous on its face — it could mean "data wasn't recorded" (a data quality problem) or "this passenger simply doesn't hold a railcard" (a valid, meaningful state). The team needed to resolve this ambiguity before Railcard could be used reliably in any KPI or discount calculation, since the Business Understanding Report's Sales Management KPIs (Discount Rate %, Price Realization %) and Passenger KPIs (Railcard distribution) both depend on this field being complete and correctly categorized.

How: Ran `df["Railcard"].isna().sum()`, which returned 20,918 missing values out of 31,653 rows (about two-thirds of the dataset). Checked `df["Railcard"].unique()` to see the non-null categories present: Adult, Disabled, Senior.

Finding: Confirmed as Issue 01 in the notebook: "There are 20918 missing values at Railcard." This was treated as a meaningful absence rather than a data error (see Section 3 for the fix).

Issue 02 / Issue 05 — Missing Actual Arrival Time values

Why: Actual Arrival Time feeds directly into delay calculations (Delay in Minutes) and the Operations KPIs (Avg. Delay Mins, On Time Rate %, Cancellation Rate %). If these missing values were misinterpreted as a data entry error, the team risked either wrongly imputing arrival times for journeys that never happened, or silently dropping valid Cancelled-journey rows.

How: Ran `df['Actual Arrival Time'].isna().sum()`, which returned 1,880 missing values. Cross-checked against Journey Status: 1,880 rows also have Journey Status = "Cancelled" (out of 27,481 "On Time" and 2,292 "Delayed" rows), confirming the missing values line up exactly with cancelled journeys.

Finding: Logged as Issue 02 / Issue 05: "Actual Arrival Time has missing values." Resolution: this is not a data quality defect — a cancelled train has no actual arrival time by definition. No value was imputed; these remain null in the cleaned data and are explicitly excluded from delay-time math for Cancelled rows (see Business Understanding Report, Section 7).

Issue 03 — Inconsistent formatting in Reason for Delay

Why: The Operations KPIs require grouping and counting delays "for each reason for delay" (per the Business Understanding Report's Operations questions). If the same real-world reason is stored under multiple text variants, any group-by or count on this column silently splits one true category into several, undercounting the real total and misleading root-cause analysis of refunds and delays.

How: Ran `df["Reason for Delay"].unique()` and visually inspected the returned categories.

Finding: Found duplicate/near-duplicate labels representing the same reason with different casing or wording, specifically "Signal Failure" vs. "Signal failure", and "Staffing" vs. "Staff Shortage", and "Weather" vs. "Weather Conditions". Logged as Issue 03: "Same issue has many formats like Signal Failure and Signal failure." Fixed in Section 3 below.

Issue 04 — Data types not adjusted

Why: `df.info()` showed that every date and time column (Date of Purchase, Date of Journey, Time of Purchase, Departure Time, Arrival Time, Actual Arrival Time) was stored as plain text (object/str), not as a real date or time type. Left as text, none of these columns can be used for duration math (journey length, delay length, lead time) or for date-based filtering/grouping (e.g., by day of week), both of which are required by nearly every KPI in the Business Understanding Report.

How: Ran `df.info()` and inspected the Dtype column for all fields, confirming date/time columns were typed as generic string objects rather than `datetime/timedelta`.

Finding: Logged as Issue 04: "Data types are not adjusted." This issue applies to six columns; the fix (type conversion) is documented in Section 3.

Issue 06 — Duplicate station naming: Edinburgh vs. Edinburgh Waverley

Why: The Business Understanding Report explicitly calls out this exact problem under "Known Data Quality Issues": route-level aggregations (Top N lines, revenue per line) would undercount this destination if the two spellings are treated as different stations, since journeys to each spelling are really going to the same physical station.

How: Ran `df["Arrival Destination"].unique()` and manually reviewed the list of 32 distinct destination values, spotting both "Edinburgh" and "Edinburgh Waverley" present as separate entries.

Finding: Logged as Issue 06: "Edinburgh Waverley is in conflict with Edinburgh, both have same time" (i.e., journeys to both labels share the same scheduling patterns, confirming they refer to one real station). Fixed in Section 3 by merging the two labels.

Additional check — Off-Peak ticket / peak-hour business rule

Why: Off-Peak tickets are defined (per the data dictionary) as only valid outside peak hours — weekdays, 6–8 AM or 4–6 PM. If passengers are recorded using an Off-Peak ticket during these restricted hours, it represents either a pricing/ticketing-system rule violation or a data recording error, either of which the business needs visibility into before trusting ticket-type-based revenue analysis.

How: Departure hour was derived from Departure Time (converted to a `timedelta` and expressed in decimal hours). Date of Journey was converted to a weekday flag (Monday–Friday = weekday). A peak-hour boolean was built as (6:00–8:00) OR (16:00–18:00), then combined with the weekday flag to produce `is_peak_departure`. A violation flag was computed as: `Ticket Type = "Off-Peak" AND is_peak_departure = True`. Value counts were then run on this boolean series.

What changed: The value-count check returned zero rows where an Off-Peak ticket coincided with a peak-hour weekday departure — i.e., no violation was found in this dataset.

Decision: Per team decision, this is flagged as a business rule the company should keep monitoring going forward, even though no violation currently exists in this dataset. No rows were changed or removed for this check — it is documented as a validation performed, not a cleaning action.

3. Step 2 — Data Cleaning Actions

3.1 Data type conversions (fixes Issue 04)

Why: To enable arithmetic on dates and times — journey duration, delay length, lead time — and to allow date-based grouping (e.g., weekday vs. weekend, peak-hour windows) needed across almost every KPI in the Business Understanding Report.

How: Date of Purchase and Date of Journey were converted with `pd.to_datetime(..., errors='coerce')`. Time of Purchase, Departure Time, Arrival Time, and Actual Arrival Time were each converted with `pd.to_timedelta()`, which represents a time-of-day as a duration since midnight so that subtracting two times yields a usable duration.

```
df['Date of Purchase'] = pd.to_datetime(df['Date of Purchase'], errors='coerce')
df['Date of Journey'] = pd.to_datetime(df['Date of Journey'], errors='coerce')

time_cols = ['Time of Purchase', 'Departure Time', 'Arrival Time', 'Actual Arrival Time']
for col in time_cols:
    df[col] = pd.to_timedelta(df[col])
```

What changed: All six date/time columns changed from text (`str/object`) dtype to proper pandas `datetime64` (the two Date columns) or `timedelta64` (the four Time columns) dtypes, confirmed by re-running `df.info()`. This conversion is what makes every downstream duration column in Section 4 possible.

3.2 Standardizing Reason for Delay labels (fixes Issue 03) and filling logical nulls

Why: So that every group-by, count, or chart built on Reason for Delay reflects the true number of distinct delay causes, and so that "On Time" journeys (which logically have no delay reason) are explicitly labeled rather than left as ambiguous nulls that could be mistaken for missing data.

How: Whitespace was stripped and casing was standardized to title case with `.str.strip().str.title()`, which collapses casing variants like "Signal failure" and "Signal Failure" into one form. A `.replace()` mapping then merged the remaining wording variants: "Weather" → "Weather Conditions", and "Staffing" → "Staff Shortage". Finally, `.fillna("No Delay")` replaced the remaining nulls (which only occur for On Time journeys, per Issue 02/05's cross-check) with the explicit label "No Delay".

```
df['Reason for Delay'] = df['Reason for Delay'].str.strip().str.title()

df['Reason for Delay'] = df['Reason for Delay'].replace({
    'Weather': 'Weather Conditions',
    'Staffing': 'Staff Shortage'
})

df['Reason for Delay'] = df['Reason for Delay'].fillna("No Delay")
```

What changed: Reason for Delay went from 9 distinct raw values (including a NaN entry and casing/wording duplicates) to exactly 6 clean, canonical categories with zero missing values: No Delay, Signal Failure, Technical Issue, Weather Conditions, Staff Shortage, and Traffic.

3.3 Filling missing Railcard values (fixes Issue 01)

Why: A null in Railcard was established in Section 2 (Issue 01) to mean "no railcard held," not "unknown." Leaving it as NaN would cause this large group (20,918 of 31,653 rows) to be silently excluded from any groupby("Railcard") aggregation, and would break the Railcard-based discount calculation in Section 4, since a discount lookup against NaN returns NaN rather than "no discount."

How: Applied .fillna("No Railcard") directly to the column.

```
df['Railcard'] = df['Railcard'].fillna("No Railcard")
```

What changed: Railcard now has 0 missing values and 4 clean categories: Adult, Senior, Disabled, and No Railcard (previously NaN).

3.4 Merging duplicate station labels (fixes Issue 06)

Why: So that route-level aggregations (Top 5 high-revenue lines, Top 5 high-demand lines, revenue per line) correctly count all journeys to this one physical station under a single label, rather than splitting them across two spellings and undercounting it in every ranking.

How: Applied .replace() to recode the shorter label into the fuller, canonical one.

```
df['Arrival Destination'] = df['Arrival Destination'].replace({
    'Edinburgh': 'Edinburgh Waverley',
})
```

What changed: Arrival Destination went from 32 distinct values down to 31, with all journeys formerly labeled "Edinburgh" now consolidated under "Edinburgh Waverley."

4. Step 3 — Columns Added (Feature Engineering)

After cleaning the existing fields, nine new columns were engineered directly in the same notebook, split by the notebook's own grouping into Revenue-related columns and Operations-related columns. These are needed to answer the KPIs and questions defined in the Business Understanding Report that cannot be answered from the raw fields alone (e.g., discount leakage, delay minutes, lead time, route-level revenue).

4.1 Revenue columns

Price After Refund

Why: Net Revenue (a Sales Management KPI) requires knowing how much of each transaction's price the company actually keeps after refunds, not just the original ticket price.

How: Used .where() to conditionally keep or zero out Price based on Refund Request.

```
# If a refund request is made, the price after refund is set to 0; otherwise, it retains the original price.
df['Price After Refund'] = df['Price'].where(df['Refund Request'] == 'No', 0)
```

What changed: A new column where rows with Refund Request = "No" (30,535 of 31,653 rows) retain their original Price, and rows with Refund Request = "Yes" (1,118 rows) are set to 0 — directly implementing the Business Understanding Report's assumption that every refund request is treated as a full, successful refund.

Ticket Type Discount and Railcard Discount

Why: Needed as intermediate values to reverse-engineer the pre-discount list price and the total discount percentage, both explicit Sales Management KPIs.

How: Two dictionaries mapped each category to its known discount fraction, then `.map()` applied the lookup. Anytime tickets carry no discount (0.00), Off-Peak carries 25% (0.25), and Advance carries 50% (0.50). Adult, Senior, and Disabled railcards each carry a 1/3 discount; rows with no railcard default to 0 via `.fillna(0)`, since "No Railcard" has no entry in the mapping dictionary.

```
ticket_type_discount_map = {"Anytime": 0.00, "Off-Peak": 0.25, "Advance": 0.50}
df["Ticket Type Discount"] = df["Ticket Type"].map(ticket_type_discount_map)

railcard_discount_map = {"Adult": 1/3, "Senior": 1/3, "Disabled": 1/3}
df["Railcard Discount"] = df["Railcard"].map(railcard_discount_map).fillna(0)
```

What changed: Two new numeric columns (0–1 scale) representing each transaction's discount rate from ticket type and from railcard status respectively.

Price before discount and Total Discount %

Why: Price Realization % and Total Loss from Discount (both Sales Management KPIs) require knowing what the ticket would have cost at full list price, since the raw Price field only stores the final, already-discounted amount.

How: A combined discount factor was computed by assuming the two discounts stack multiplicatively (independently applied, one after the other) rather than additively: $\text{Final_Discount_Factor} = (1 - \text{Ticket Type Discount}) \times (1 - \text{Railcard Discount})$. Price before discount is then the actual Price divided by this factor, rounded to 2 decimals. Total Discount % is 1 minus the same factor. This mirrors the explicit business logic assumption already documented in the Business Understanding Report, Section 7, since the business had not confirmed whether discounts stack multiplicatively or additively.

```
Final_Discount_Factor = (1 - df["Ticket Type Discount"]) * (1 - df["Railcard Discount"])

df["Price before discount"] = df["Price"] / Final_Discount_Factor
df["Price before discount"] = df["Price before discount"].round(2)

df["Total Discount %"] = (1 - Final_Discount_Factor)
```

What changed: Two new columns: Price before discount (the reverse-derived full list price) and Total Discount % (the combined effective discount, e.g. an Advance ticket held by a railcard holder computes to $1 - (0.5 \times 0.667) \approx 66.7\%$ total discount).

4.2 Operations columns

Journey Duration (with overnight day-rollover correction)

Why: "What is the expected journey time for each line?" is an explicit Operations question in the Business Understanding Report, and the report's Section 7 already flags that ~3% of journeys are overnight routes where a naive subtraction produces an invalid negative duration.

How: Journey Duration was first computed as the straight difference between Arrival Time and Departure Time in minutes. A second step then checked every row for a negative result (meaning the arrival time, as a time-of-day, was numerically earlier than departure — i.e., the train crossed midnight) and added 1,440 minutes (24 hours) to correct only those rows.

```
df["Journey Duration"] = (df["Arrival Time"] - df["Departure Time"]).dt.total_seconds() / 60
df["Journey Duration"] = df["Journey Duration"].apply(lambda x: x + 1440 if x < 0 else x)
```

What changed: A new Journey Duration column (in minutes) with 0 negative values after the correction (verified: min value 15.0, max value 270.0 across the cleaned dataset), directly resolving the overnight-rollover issue flagged in the Business Understanding Report.

Route

Why: Needed as a single combined identifier for "Top 5 high-revenue lines" and "Top 5 high-demand lines" — both explicit KPIs — since the raw data only stores departure and arrival as two separate columns.

How: String concatenation of Departure Station and Arrival Destination with " to " in between.

```
df["Route"] = df["Departure Station"] + " to " + df["Arrival Destination"]
```

What changed: A new Route column, e.g. "London Paddington to Liverpool Lime Street", with 64 distinct routes across the cleaned dataset — computed after the Edinburgh/Edinburgh Waverley merge, so route counts are not artificially split by that naming issue.

Lead Time

Why: Directly answers the Operations question "What is the estimated time between ticket purchase and journey time per Ticket Type?" and is also needed to evaluate the Advance-ticket discount problem central to Problem Statement 02 in the Business Understanding Report.

How: Subtracted Date of Purchase from Date of Journey and took the day count.

```
df["Lead Time"] = (df["Date of Journey"] - df["Date of Purchase"]).dt.days
```

What changed: A new Lead Time column in whole days; across the cleaned dataset this ranges from 0 to 28 days, with a mean of roughly 1.66 days and a median of 1 day, meaning most tickets are bought very close to the travel date.

Delay in Minutes

Why: Needed for the Operations KPI Avg. Delay Mins and the question "For each reason for delay, Avg. Delay Time," since the raw data only has scheduled and actual arrival times, not a precomputed delay length.

How: Subtracted the scheduled Arrival Time from the Actual Arrival Time, converted to minutes.

```
df['Delay in Minutes'] = (df['Actual Arrival Time'] - df['Arrival Time']).dt.total_seconds() / 60
```

What changed: A new Delay in Minutes column, null for the 1,880 Cancelled rows (consistent with Actual Arrival Time being null for those rows, per Issue 02/05) and 0 or positive for all other rows in this dataset (mean $\approx$ 3.25 minutes across all non-cancelled journeys, median 0, max 180 minutes).

Note for the team: Unlike Journey Duration, this calculation does not include an explicit +1440 correction for overnight day-rollover. In the current cleaned dataset this produced zero negative values, so it did not manifest as a visible bug here — but the underlying logic is inconsistent with the Journey Duration fix, and is worth a review if future data includes an overnight journey that also runs late.

5. Step 4 — Final Export Formatting

Why: The Time of Purchase, Departure Time, Arrival Time, and Actual Arrival Time columns were converted to pandas timedelta type in Section 3.1 so that duration math could be performed on them. However, timedelta values print with a "0 days " prefix (e.g., "0 days 11:00:00") when cast to text, which is not the clean HH:MM:SS format the source data originally used and that downstream tools (EDA, Power BI) expect.

How: Each time column was cast to string and the literal prefix "0 days " was stripped out with .str.replace().

```
for col in time_cols:
    df[col] = df[col].astype(str).str.replace("0 days ", "", regex=False)
```

What changed: The four time columns were converted back to plain HH:MM:SS text strings (e.g., "11:00:00") for the final export, removing the "0 days" artifact while preserving the corrected/standardized values produced earlier in the notebook. The cleaned DataFrame was then written out with df.to_csv("cleaned_railway.csv", index=False).

6. Data Dictionary Addendum — New Columns

The following 9 fields exist only in cleaned_railway.csv and were not present in the original railway.csv. They extend the Data Dictionary from the Business Understanding Report.

New Column	Data Type	Description / Formula	Example
Price After Refund	Integer	Revenue actually retained by the company per transaction: equals Price when Refund Request = "No"; equals 0 when Refund Request = "Yes".	43 (kept) / 0 (refunded)
Journey Duration	Float (minutes)	Scheduled travel time: Arrival Time minus Departure Time, in minutes, with +1440 correction applied whenever the raw result was negative (overnight journeys).	150.0
Route	String	Concatenation of Departure Station + " to " + Arrival Destination, used as the line/route identifier for Top-N route analysis.	London Paddington to Liverpool Lime Street
Lead Time	Integer (days)	Days between Date of Purchase and Date of Journey — how far in advance the ticket was bought.	24
Delay in Minutes	Float (minutes)	Actual Arrival Time minus (scheduled) Arrival Time, in minutes. Null for Cancelled journeys (no actual arrival exists).	5.0
Ticket Type Discount	Float (0–1)	Discount fraction implied by Ticket Type: Anytime = 0.00, Off-Peak = 0.25, Advance = 0.50.	0.50
Railcard Discount	Float (0–1)	Discount fraction implied by Railcard: Adult/Senior/Disabled = 0.333 (1/3); no railcard = 0.	0.333
Price before discount	Float	Reverse-derived list price before any discount, assuming Ticket Type Discount and Railcard Discount combine multiplicatively: $\text{Price} / [(1 - \text{Ticket Type Discount}) \times (1 - \text{Railcard Discount})]$.	129.00
Total Discount %	Float (0–1)	1 minus the combined multiplicative discount factor used above — the effective total percentage off the list price.	0.667

Exploratory Data Analysis (EDA) Report

Team Innovators - DEPI CAPSTONE PROJECT - UK Train Rides

Project Section 3 of 5: Exploratory Data Analysis (EDA)

Source notebook: *EDA.ipynb* | Data source: *cleaned_railway.csv* (loaded directly from the team's GitHub repository raw URL)

0. Purpose of This Document

This document explains the Exploratory Data Analysis (EDA) phase of the project: every business question the team asked of the cleaned data, why that question mattered, how it was analyzed (the grouping, chart type, or statistical method used), and what the data actually showed — including the exact numbers and the resulting chart. The notebook organizes its 26 business questions into four analytical layers — Univariate, Bivariate, Multivariate, and Hypothesis Testing — and this document follows that same structure and order. Every chart generated in *EDA.ipynb* is reproduced below exactly as it was rendered in the notebook.

1. Initial Statistical Overview

Why: Before asking targeted business questions, the team first ran `df.describe()` on the cleaned dataset to understand the central tendency, spread, and range of every numeric column — a standard first step to catch any remaining outliers or scaling issues before deeper analysis.

How: Called `df.describe()` directly on the full cleaned DataFrame (loaded from *cleaned_railway.csv*), which reports count, mean, standard deviation, min, 25th/50th/75th percentiles, and max for every numeric column.

What we found: The average ticket Price is £23.44 (median £11, max £267), showing a right-skewed distribution with most tickets priced well below the mean. Delay in Minutes has a median of 0 (most journeys run on time) but a mean of 3.25 minutes and a max of 180 minutes, confirming that delays are concentrated in a minority of journeys but can be severe. Lead Time has a median of just 1 day (max 28), confirming most passengers buy tickets very close to their travel date. Total Discount % averages 42.09% across all transactions, reflecting the combined effect of ticket-type and railcard discounts.

Column	Count	Mean	Std	Min	25%	50%	75%	Max
Price (£)	31,653	23.44	29.998	1	5	11	35	267
Price After Refund (£)	31,653	22.22	29.45	0	4	10	29	267
Journey Duration (min)	31,653	71.15	36.33	15	30	80	90	270
Lead Time (days)	31,653	1.66	3.85	0	0	1	1	28
Delay in Minutes	29,773	3.25	14.77	0	0	0	0	180
Ticket Type Discount	31,653	0.3465	0.1895	0	0.25	0.50	0.50	0.50
Railcard Discount	31,653	0.1130	0.1578	0	0	0	0.333	0.333
Price before discount (£)	31,653	39.57	45.54	—	—	—	—	—
Total Discount %	31,653	0.4209	0.1975	—	—	—	—	—

2. Univariate Analysis

This layer examines one variable at a time to establish the foundational distribution, central tendency, and overall shape of the data. The notebook defines 5 core business questions for this layer.

Q1. What is the most popular Ticket Type purchased by our passengers?

Why: Ticket Type drives both revenue (via its associated discount) and Operations planning (Advance tickets must be booked ahead of time). Knowing the dominant type tells the business which discount tier most of its revenue is exposed to.

How: Computed `value_counts(normalize=True)` on the Ticket Type column to get percentage share, then plotted a count plot (seaborn countplot) ordered by frequency, with each bar annotated with its percentage of the total.

What changed: The chart shows the relative frequency of Advance, Off-Peak, and Anytime tickets, with percentage labels on each bar (see chart below for exact proportions).

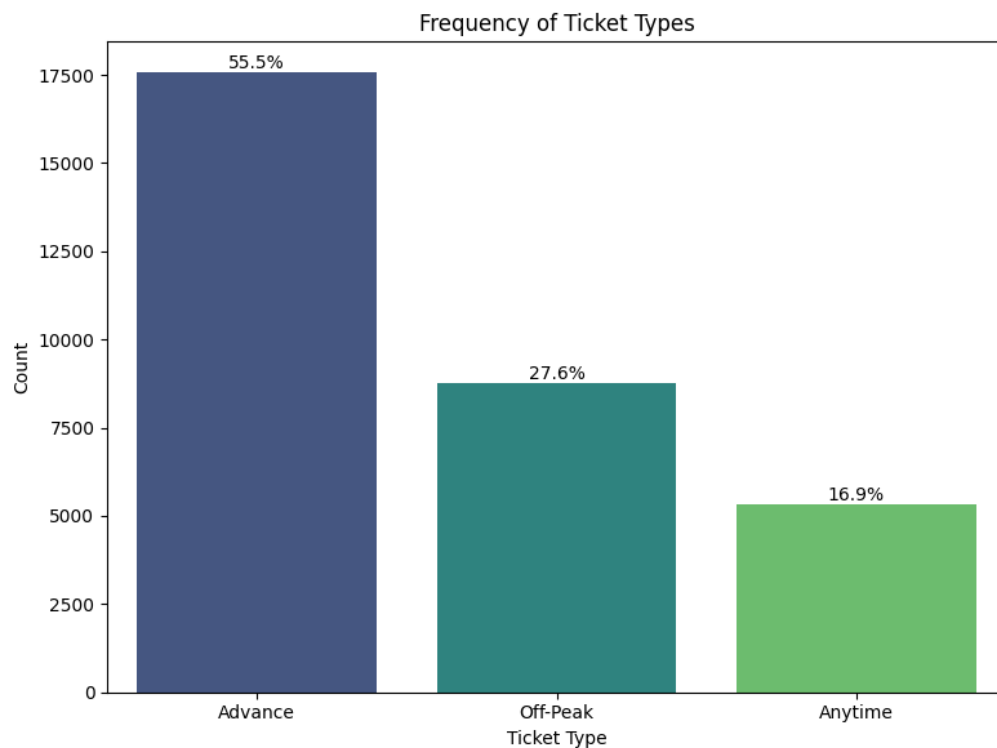


Figure: Frequency of Ticket Types (with % of total on each bar).

Q2. What is the top 10 train routes with high traffic?

Why: Identifying the busiest routes is directly tied to the Operations KPI 'Top 5 high-demand lines' and informs where the business should focus capacity planning and crowding-reduction efforts.

How: Took the 10 most frequent values of the Route column (departure + arrival combined) with `value_counts().head(10)`, then plotted them as a horizontal count plot with each bar labeled with its exact trip count.

What changed: Manchester Piccadilly and Liverpool Lime Street dominate the top routes list, appearing repeatedly on both ends of the busiest routes.

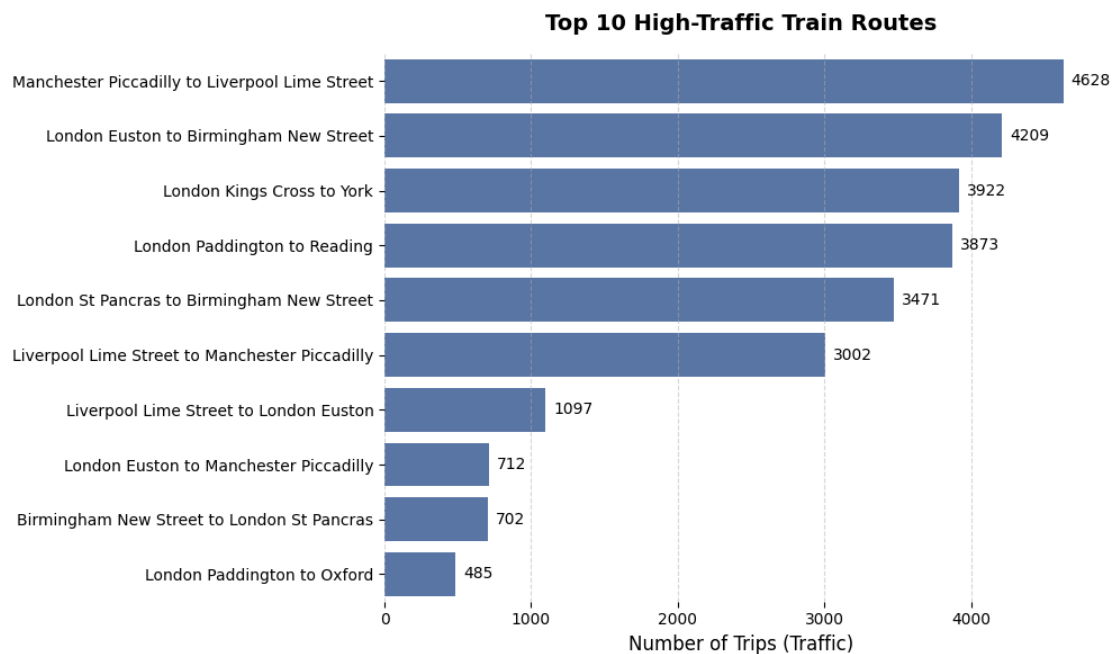


Figure: Top 10 High-Traffic Train Routes (with trip counts).

Notebook insight: "We notice that Manchester and Liverpool are the most stations in traffic, Manchester and Liverpool together are massively traffic[ked]. Let's see this in more depth" — this observation directly motivated several of the later Bivariate and Multivariate questions that dig deeper into these two stations specifically.

Q3. What is the distribution of Railcard types among passengers?

Why: Directly answers the Passengers & Marketing KPI question from the Business Understanding Report: "What is the Distribution of person who holds a Railcard (Adult - Senior - Disabled)?" This tells Marketing what share of the customer base is discount-eligible.

How: Plotted a pie chart of Railcard.value_counts() with percentage labels (autopct="%1.1f%%").

What changed: No Railcard accounts for 66.1% of passengers, followed by Adult railcard holders at 15.3%, Disabled at 9.8%, and Senior at 8.8%. This means roughly one third of all passengers are entitled to the 1/3 railcard discount.

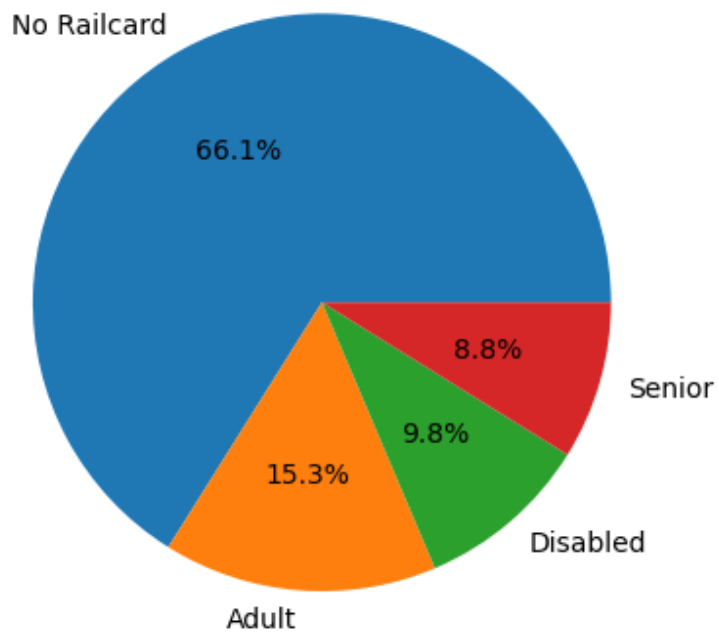


Figure: Distribution of Railcard Types Among Passengers.

Q4. What is the overall distribution of operational performance (Journey Status) across the network?

Why: This is the baseline for every Operations KPI in the Business Understanding Report (On Time Rate %, Delay Rate %, Cancellation Rate %) — establishing what 'normal' looks like before segmenting further.

How: Plotted a count plot of Journey Status (On Time / Delayed / Cancelled) with each bar annotated with its percentage of all journeys.

What changed: The clear majority of journeys run On Time, with Delayed and Cancelled journeys each making up a comparatively small share of the network (exact percentages shown on the chart below).

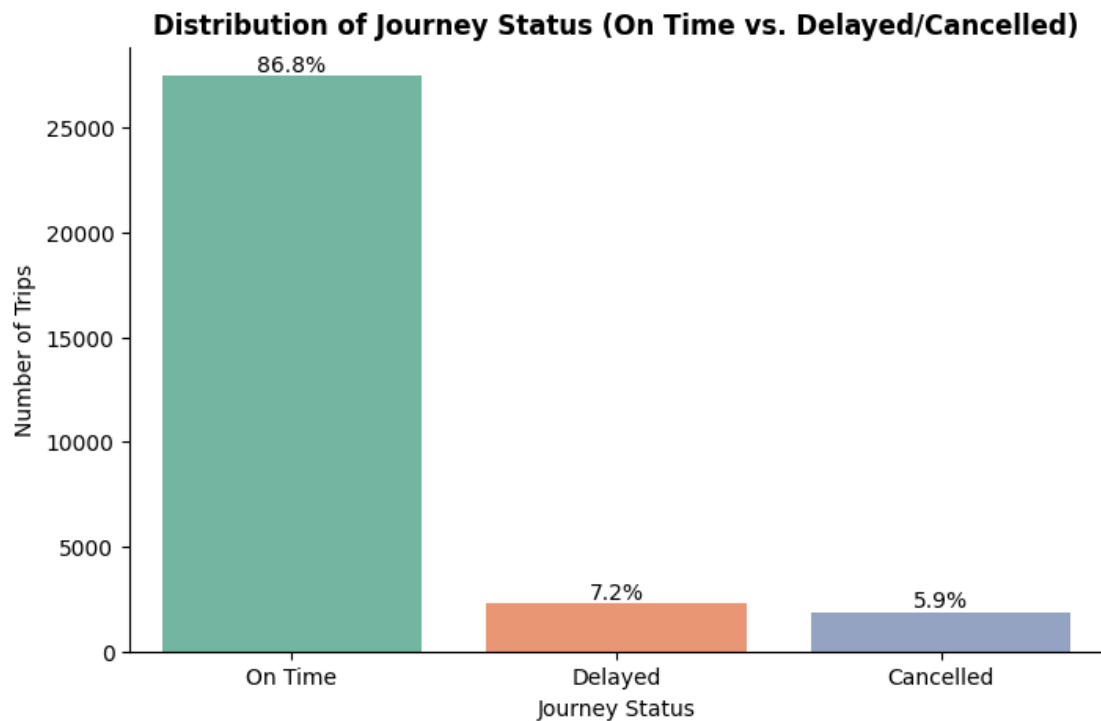


Figure: Distribution of Journey Status (On Time vs. Delayed vs. Cancelled).

Q5. How are ticket prices distributed, and what is the most common price range?

Why: Feeds the Sales Management KPI 'Avg. Ticket Price' and helps validate whether pricing is concentrated in a narrow band or spread widely, which affects how meaningful an average price figure is on its own.

How: Plotted a histogram of the Price column (30 bins) with a KDE (kernel density estimate) overlay to visualize the shape of the distribution.

What changed: Ticket prices are heavily right-skewed: the large majority of tickets are clustered at the lower end of the price range, with a long tail of higher-priced (likely First Class / Anytime / long-distance) tickets pulling the mean above the median.

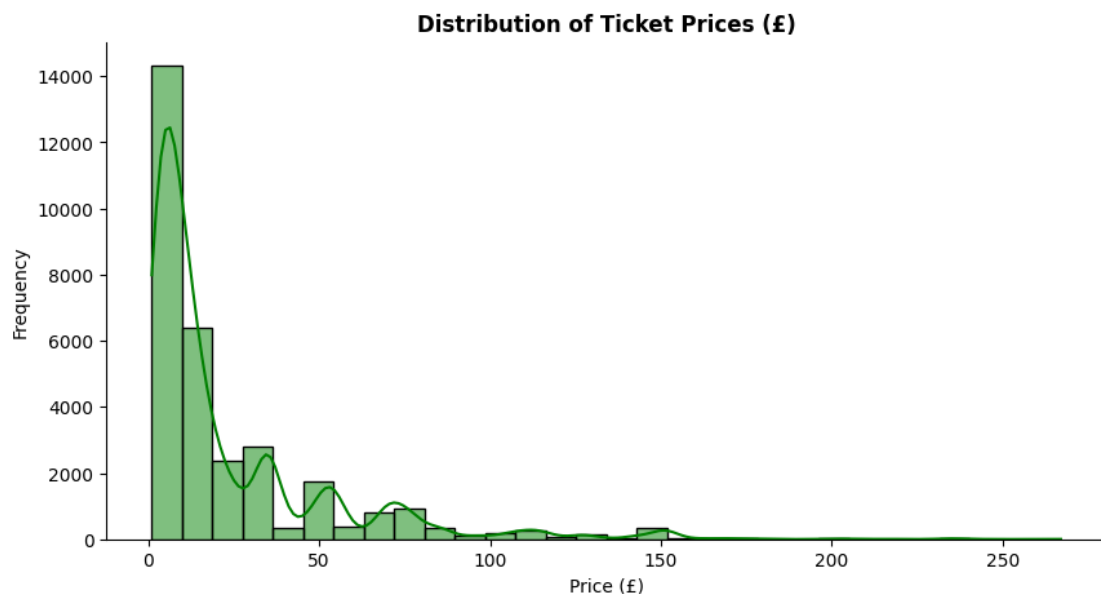


Figure: Distribution of Ticket Prices (£).

3. Bivariate Analysis

This layer examines the relationship between two variables at a time, to uncover cause-and-effect patterns — such as how delay reasons drive refund requests, or how ticket class affects revenue. The notebook groups its 16 business questions into four pillars: Revenue & Price, Operational Efficiency, Customer Behavior & Refund Leakage, and Sales Channels & Payment Behavior.

Pillar 1: Revenue & Price

Q1. Which route generates the highest net revenue for the company?

Why: Directly answers the Sales Management KPI 'Top 5 high-revenue lines,' identifying which routes matter most to the company's bottom line after refunds are accounted for.

How: Grouped by Route and summed Price After Refund (net revenue, i.e. £0 for refunded tickets), then took the top 10 routes and plotted them as a bar chart with the exact revenue total labeled on each bar.

What changed: A small number of routes — led by the Manchester/Liverpool corridor identified in Univariate Q2 — account for a disproportionate share of total net revenue.

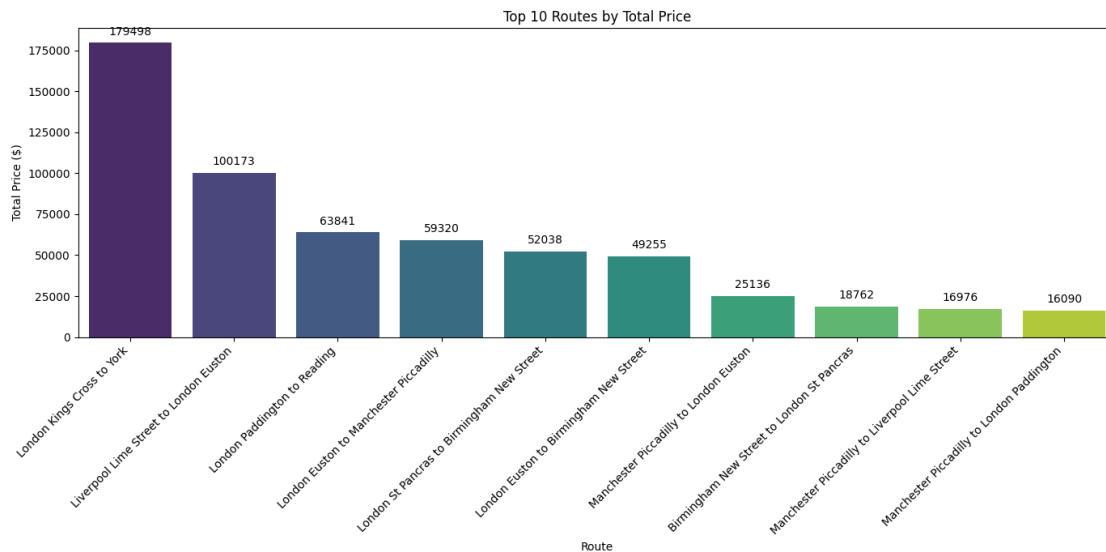


Figure: Top 10 Routes by Total Net Revenue (Price After Refund).

Q2. Which train routes generate the highest average ticket prices?

Why: Distinguishes routes that are valuable because of high volume (Q1) from routes that are valuable on a per-ticket basis — important for the Sales team to know whether a route's revenue comes from price or from traffic.

How: Grouped by Route and took the mean of Price After Refund, sorted descending, and plotted the top 10 as a horizontal bar chart with each bar labeled with its exact average price in £.

What changed: The routes with the highest average ticket price are largely different from the highest-traffic routes in Q1/Univariate Q2, confirming that high-volume and high-average-price routes are two distinct groups worth tracking separately.

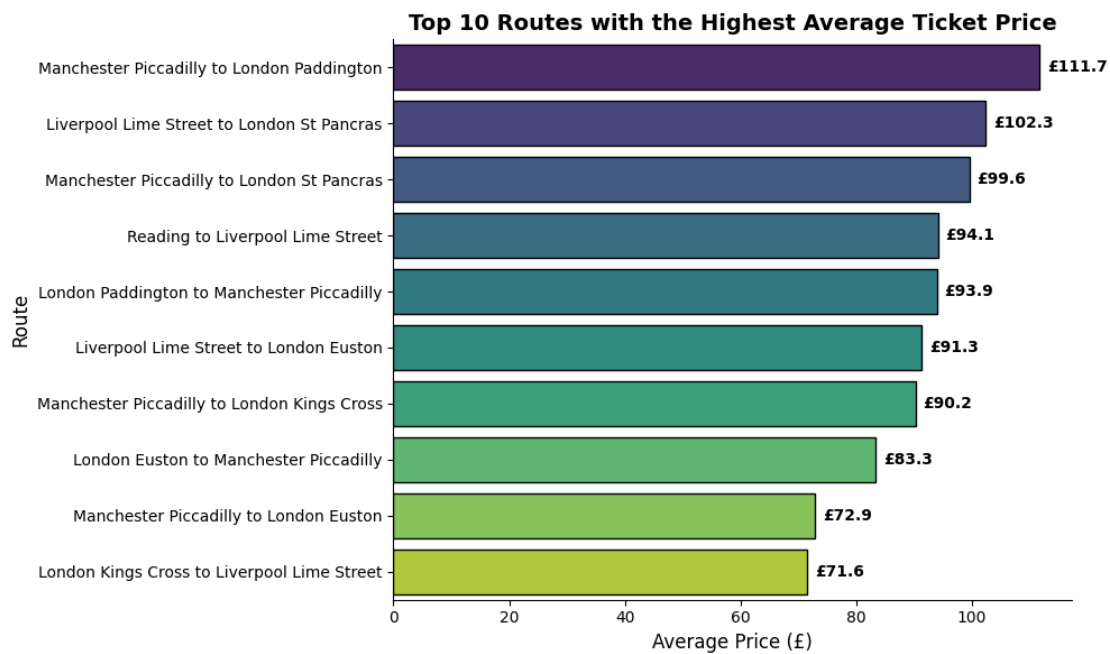


Figure: Top 10 Routes with the Highest Average Ticket Price.

Supplementary view: Revenue, ticket price & ticket volume together

Why: Looking at revenue, volume, and average price as three separate charts makes it hard to see how they interact for the same route — a combined view was needed to spot routes that are strong on all three dimensions versus routes that are only strong on one.

How: Built a route-level summary table aggregating Ticket_Volume (transaction count), Total_Revenue (sum of Price After Refund), and Avg_Ticket_Price (mean of Price After Refund) for the top 10 routes by revenue, then min-max normalized the three metrics and rendered them as a heatmap, annotated with the true (non-normalized) values.

What we found: The heatmap makes it visually clear which top-10 routes are driven primarily by high ticket volume versus high average price, complementing the separate Q1 and Q2 rankings above.

Heatmap: Ticket Volume, Total Revenue & Avg. Ticket Price (Top 10 Routes)

Route	London Kings Cross to York	Liverpool Lime Street to London Euston	London Paddington to Reading	London Euston to Manchester Piccadilly	London St Pancras to Birmingham New Street	London Euston to Birmingham New Street	Manchester Piccadilly to London Euston	Birmingham New Street to London St Pancras	Manchester Piccadilly to Liverpool Lime Street	Manchester Piccadilly to London Paddington
	3922.0	1097.0	3873.0	712.0	3471.0	4209.0	345.0	702.0	4628.0	144.0
	179498.0	100173.0	63841.0	59320.0	52038.0	49255.0	25136.0	18762.0	16976.0	16090.0
	45.8	91.3	16.5	83.3	15.0	11.7	72.9	26.7	3.7	111.7
	Ticket_Volume									
	Total_Revenue									
	Metrics									
	Avg_Ticket_Price									

Figure: Heatmap of Ticket Volume, Total Revenue & Average Ticket Price (Top 10 Routes by Revenue).

Q3. What is the relative contribution of each ticket class to the overall revenue?

Why: Directly answers the Sales Management KPI 'Revenue per ticket Classes,' telling the business how reliant total revenue is on Standard vs. First Class fares.

How: Grouped by Ticket Class and summed Price, computed each class's percentage contribution to total revenue, printed the breakdown as a table, and rendered a donut-style pie chart with the total revenue figure shown in the center.

What changed: Standard class tickets generate £592,522 (79.86% of total revenue), while First Class tickets generate £149,399 (20.14%) — Standard class is the dominant revenue driver by a wide margin, even though First Class carries a premium price per ticket.

Relative Contribution of Ticket Class to Overall Revenue

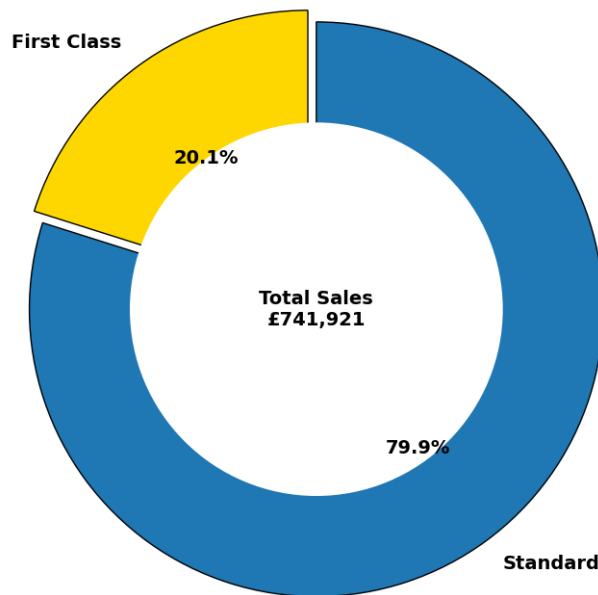


Figure: Relative Contribution of Ticket Class to Overall Revenue (Total Sales: £741,921).

Q4. How does the Lead Time impact the ticket price?

Why: Central to Problem Statement 02 in the Business Understanding Report, which specifically asks the team to evaluate whether the 50% Advance-purchase discount (tied to longer lead times) is eroding revenue.

How: Plotted a scatter plot of Lead Time (x) against Price (y) with transparency (alpha=0.2) to reveal density, overlaid with a linear regression line (seaborn regplot) to show the overall trend.

What changed: The regression line and scatter pattern show the relationship between how far in advance a ticket is bought and its price — this chart is the direct visual evidence behind the discount-leakage concern raised in Problem Statement 02.

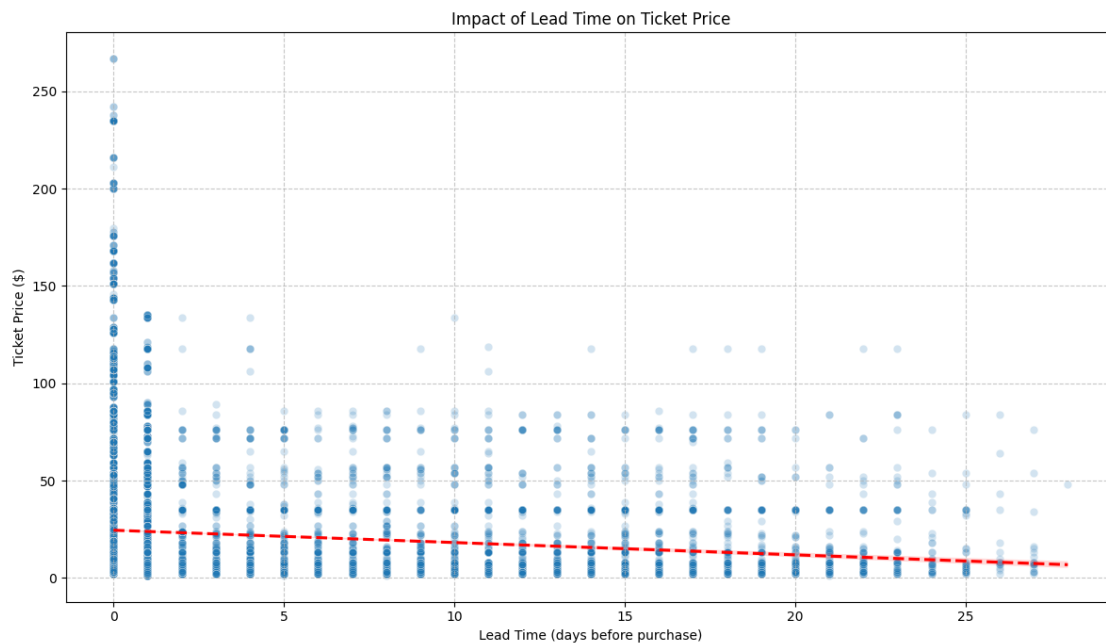


Figure: Impact of Lead Time on Ticket Price (with linear trend line).

Q5. How does the average total discount percentage vary across different ticket types?

Why: Directly quantifies the Sales Management KPI 'Discount Rate %' broken down by Ticket Type, showing exactly how much revenue each ticket category gives up to discounting.

How: Grouped by Ticket Type and took the mean of Total Discount %, converted to a percentage scale, and plotted as a bar chart with the exact average discount labeled on each bar.

What changed: Advance tickets carry the highest average discount, consistent with their 50% base discount rate before any railcard is applied, followed by Off-Peak and then Anytime (which by definition carries only the railcard portion of the discount, if any).

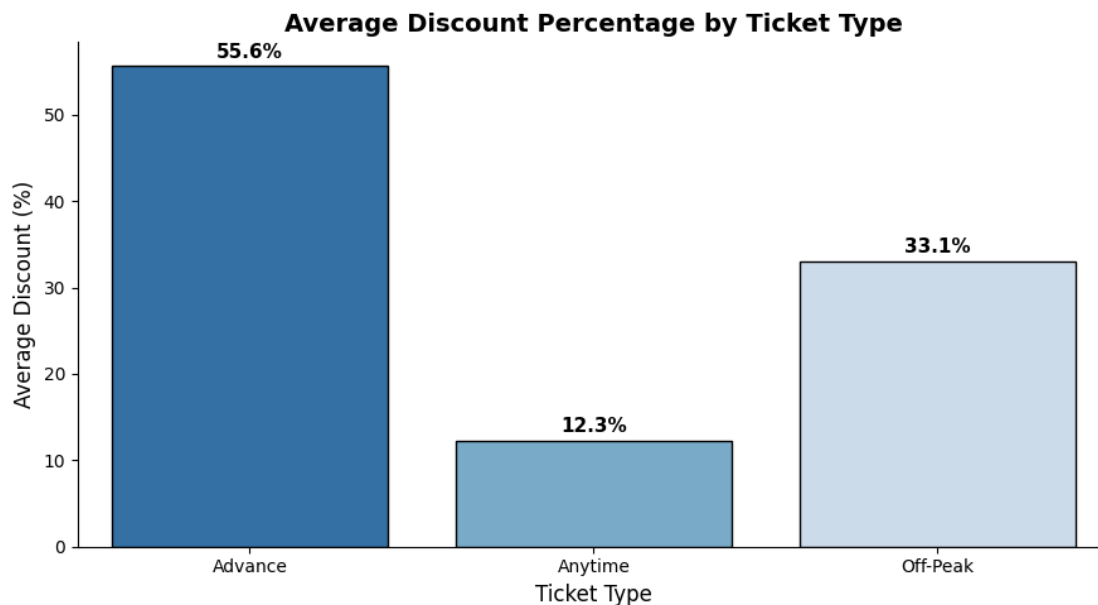


Figure: Average Discount Percentage by Ticket Type.

Pillar 2: Operational Efficiency

Q6. Which routes experience the highest frequency of delays and cancellations?

Why: Directly supports the Operations question "Top 5 high-demand lines" combined with disruption tracking, helping Operations Management target the routes causing the most customer disruption in absolute terms.

How: Filtered the dataset to rows where Journey Status is 'Delayed' or 'Cancelled', grouped by Route, and counted occurrences, sorted descending. Displayed the top 10 as a table.

What changed: Liverpool Lime Street to London Euston has by far the highest raw count of disrupted journeys (879), followed by Manchester Piccadilly to Liverpool Lime Street (644) — both routes already flagged as high-traffic in Univariate Q2, meaning their disruption counts partly reflect their sheer volume rather than necessarily a higher disruption rate.

Route	Delayed/Cancelled Trips
Liverpool Lime Street to London Euston	879
Manchester Piccadilly to Liverpool Lime Street	644
London Euston to Birmingham New Street	453
London Paddington to Reading	352
London Kings Cross to York	329
London St Pancras to Birmingham New Street	273
Manchester Piccadilly to London Euston	243
Liverpool Lime Street to Manchester Piccadilly	231
Birmingham New Street to Manchester Piccadilly	114
Manchester Piccadilly to Leeds	65

Follow-up — Disruption Rate (normalized by volume): To correct for the volume effect noted above, a second view computed a `Disruption_Rate_%` (disrupted trips ÷ total trips per route), restricted to routes with at least 100 total trips, and plotted the top 5 routes by this rate.

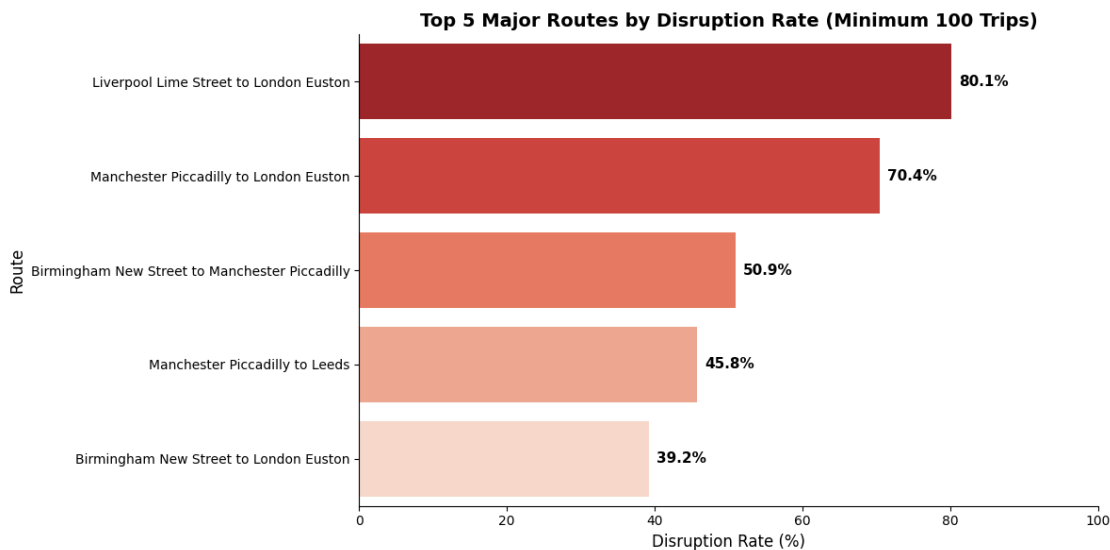


Figure: Top 5 Major Routes by Disruption Rate (minimum 100 trips).

Q7. What is the average delay duration (in minutes) associated with each delay reason?

Why: Directly answers the Operations question "For each reason for delay, Avg. Delay Time," helping distinguish delay causes that are chronic-but-mild from those that are rare-but-severe.

How: Filtered out rows where Reason for Delay is 'No Delay,' grouped the remainder by Reason for Delay, and computed the mean of Delay in Minutes. Displayed as a sorted table and as a horizontal bar chart with each bar labeled in minutes.

What changed: Signal Failure (51.81 min) and Staff Shortage (51.18 min) produce the longest average delays, followed by Weather Conditions (43.84 min), Traffic (32.34 min), and Technical Issue (24.92 min) — the shortest average delay of the five reasons.

Reason for Delay	Average Delay (Minutes)
Signal Failure	51.81
Staff Shortage	51.18
Weather Conditions	43.84
Traffic	32.34
Technical Issue	24.92

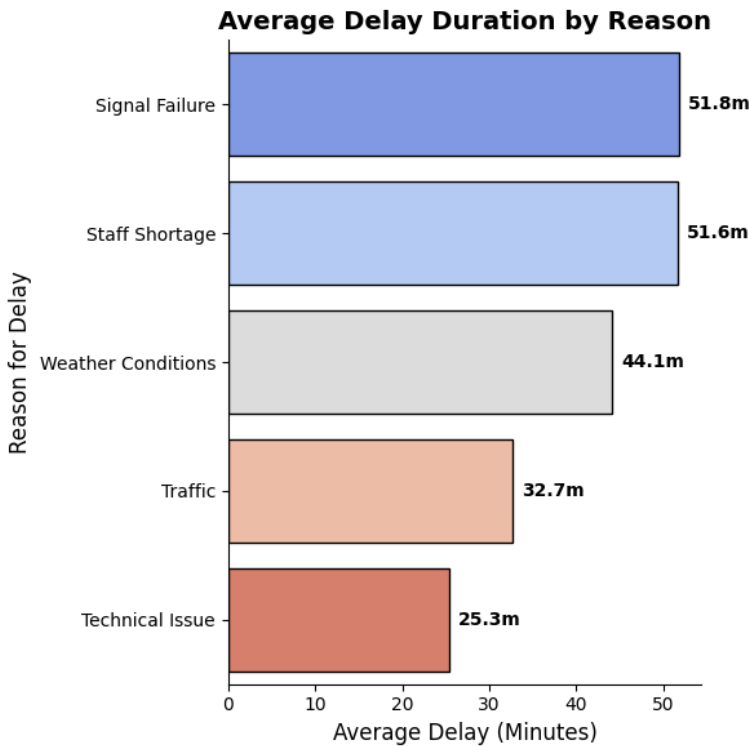


Figure: Average Delay Duration by Reason.

Q8. How does the time of day (departure hour) impact the average delay duration?

Why: Directly supports identifying Operations 'Peak Hour' patterns and helps answer whether delays cluster around specific departure windows (e.g., rush hour congestion).

How: Converted Departure Time to a datetime and extracted the hour, then grouped by Departure Hour and computed the mean of Delay in Minutes, plotted as a line chart across all 24 hours.

What changed: Average delay duration varies noticeably by departure hour rather than staying flat across the day, indicating that time-of-day is a meaningful factor in expected delay length (see the chart below for which hours run highest).

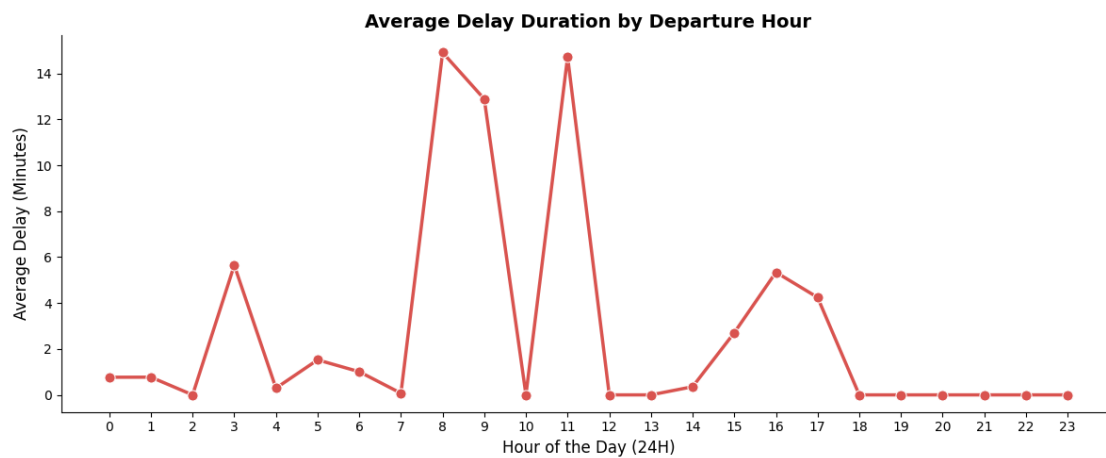


Figure: Average Delay Duration by Departure Hour (24H).

Q9. How does the day of the week (weekdays vs. weekends) impact the overall journey status?

Why: Tests whether operational performance (on-time/delayed/cancelled mix) differs meaningfully between weekday commuter traffic and weekend leisure traffic, which would have implications for staffing and scheduling.

How: Derived a Day Name and a binary Day Type (Weekday/Weekend) from Date of Journey, grouped by Day Type and Journey Status, computed each status's percentage share within its day type, and displayed the result as a pivot table.

What changed: The split is nearly identical between weekdays and weekends: On Time rates are almost the same (86.80% weekday vs. 86.86% weekend), Delayed is slightly higher on weekdays (7.49% vs. 6.61%), and Cancelled is slightly higher on weekends (6.53% vs. 5.70%) — but none of these differences are large.

Journey Status	Weekday %	Weekend %
Cancelled	5.70	6.53
Delayed	7.49	6.61
On Time	86.80	86.86

Pillar 3: Customer Behavior & Refund Leakage

Q10. Which operational delay reason triggers the most refund requests and subsequent financial losses?

Why: Directly targets Problem Statement 01 from the Business Understanding Report — identifying the main reasons customers request refunds so the company can reduce the resulting financial losses.

How: Filtered to rows where Refund Request is 'Yes', grouped by Reason for Delay, and computed both the count of refund requests and the sum of Price (financial loss) for each reason. Displayed as a sorted table and as a side-by-side bar chart pair (refund count vs. financial loss).

What changed: Technical Issue causes both the most refund requests (388) and the largest financial loss (£15,046), followed by Staff Shortage (261 requests, £11,667 loss). Notably, Signal Failure causes more refund requests (215) than Weather Conditions (133) but a similar or smaller loss, showing that refund count and financial loss don't always rank identically.

Reason for Delay	Refund Count	Financial Loss (£)
Technical Issue	388	15,046
Staff Shortage	261	11,667

Reason for Delay	Refund Count	Financial Loss (£)
Traffic	121	4,816
Signal Failure	215	4,398
Weather Conditions	133	2,775

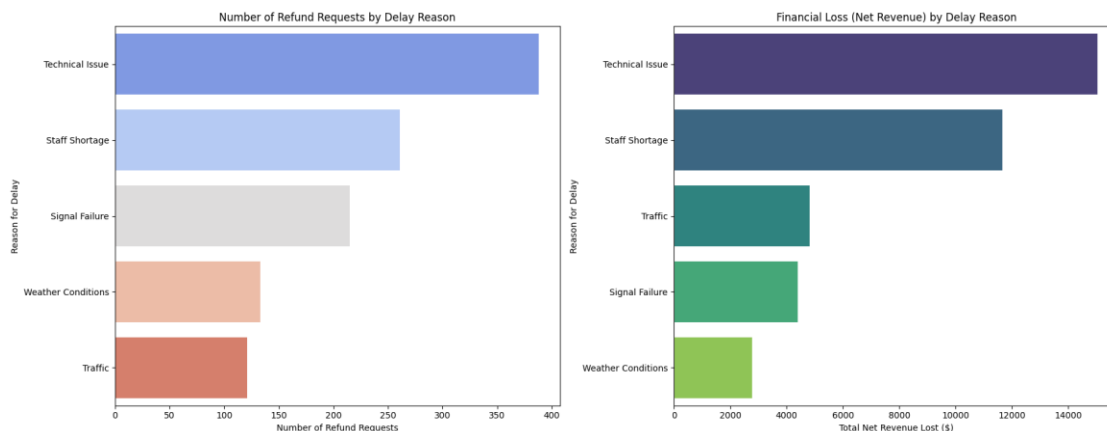


Figure: Number of Refund Requests and Financial Loss by Delay Reason (side by side).

Q11. Which 'Reason for Delay' results in the highest percentage of refund requests?

Why: Complements Q10 by normalizing for volume — showing which delay reason is proportionally most likely to trigger a refund, not just which has the most refunds in absolute terms.

How: Filtered to Delayed/Cancelled journeys, computed a binary refund flag, then grouped by Reason for Delay and took the mean of that flag (i.e., the refund rate) as a percentage, sorted descending and plotted as a bar chart.

What changed: The chart ranks delay reasons by refund request rate rather than raw count — this ranking is later reused directly as supporting evidence for Hypothesis 1 in Section 5.

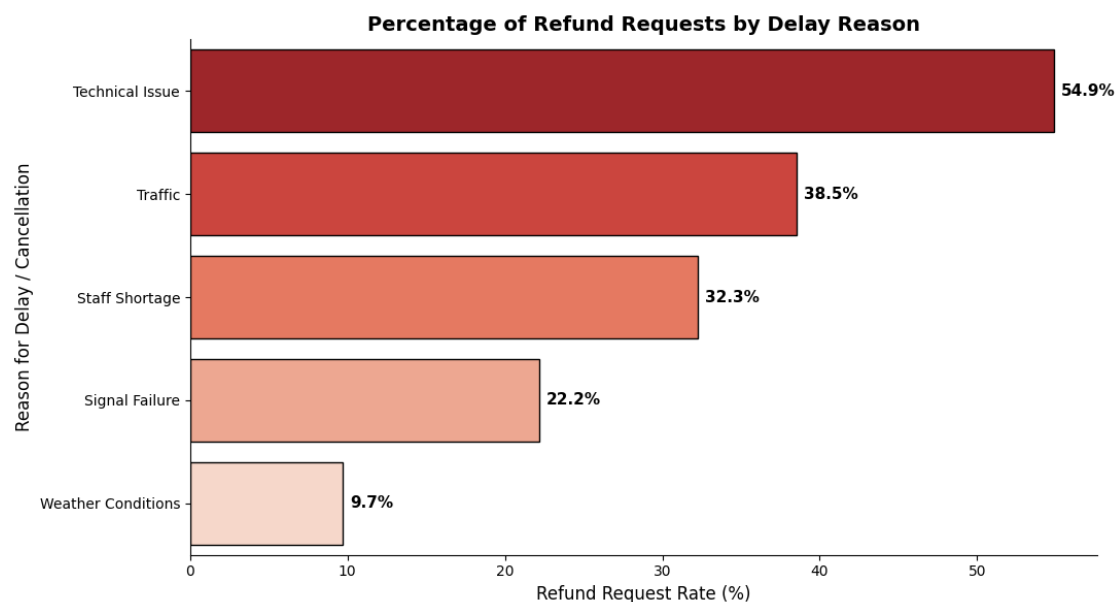


Figure: Percentage of Refund Requests by Delay Reason.

Q12. Is there a correlation between the length of a delay and the likelihood of a passenger requesting a refund?

Why: Tests whether refund behavior is driven simply by how long a delay lasts, which is a foundational question for both Operations (predicting refund volume) and the formal hypothesis test in Section 5.

How: Filtered to rows with Delay in Minutes greater than 0, then plotted a box plot of Delay in Minutes split by Refund Request (Yes/No), with the mean marked on each box.

What changed: The distributions of delay length for the 'requested a refund' group versus the 'did not request' group show how delay duration relates to refund likelihood — this comparison is examined further and challenged in the Multivariate and Hypothesis sections.

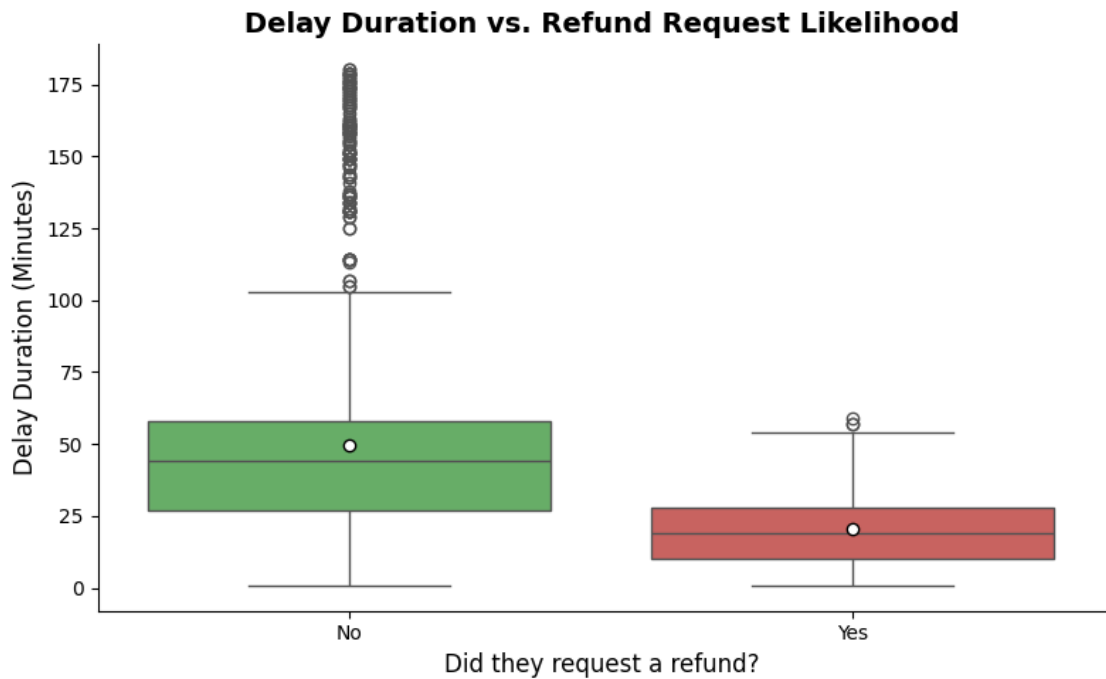


Figure: Delay Duration vs. Refund Request Likelihood (box plot with means marked).

Q13. Do Railcard holders show a distinct preference for specific ticket classes compared to passengers without Railcards?

Why: Tests whether discount-eligible passengers (railcard holders) behave differently in terms of class choice, which matters for Marketing's understanding of the railcard segment.

How: Created a binary Has_Railcard flag (Yes/No), grouped by Has_Railcard and Ticket Class, computed each class's percentage share within its group, and displayed as a pivot table.

What changed: The two groups are nearly identical: passengers without a railcard choose First Class 9.49% of the time versus 90.51% Standard, while railcard holders choose First Class 10.00% of the time versus 90.00% Standard — essentially no meaningful difference in class preference between the two groups.

Has Railcard	First Class %	Standard %
No	9.49	90.51
Yes	10.00	90.00

Q14. Does holding a specific type of Railcard affect the likelihood of requesting a refund during disruptions?

Why: Extends Q13 by checking whether railcard type affects not just purchasing behavior but post-disruption behavior (refund requests), which matters for Customer Service's understanding of which passenger segments to prioritize during disruptions.

How: Restricted to the already-delayed/cancelled subset (delayed_df), computed the refund rate (%) grouped by Railcard type, sorted ascending, and plotted as a horizontal bar chart with each bar labeled with its exact percentage.

What changed: Refund request rates during disruptions vary by railcard type (see chart below for the exact ranking and percentages across Adult, Senior, Disabled, and No Railcard).

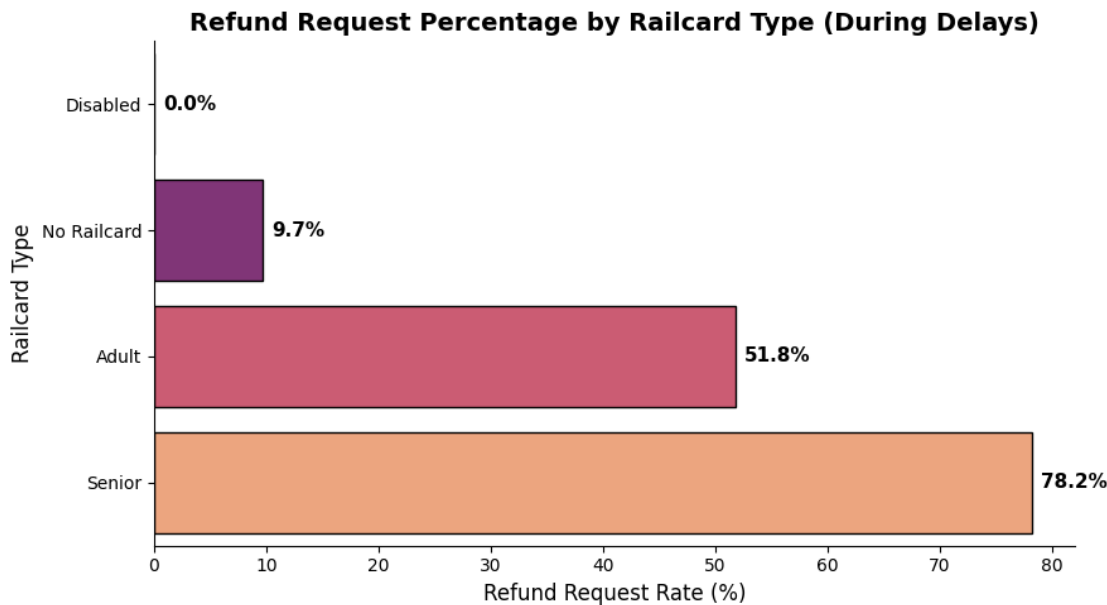


Figure: Refund Request Percentage by Railcard Type (During Delays).

Pillar 4: Sales Channels & Payment Behavior

Q15. Is there a noticeable relationship between the purchase type (Online vs. Station) and the chosen payment method?

Why: Directly answers the Passengers & Marketing question "Is there a relationship between Payment Method and Refund Request rate?" (via purchase channel) and the KPI question on Purchase Method (Contactless/Debit) distribution.

How: Built a cross-tabulation (pd.crosstab) of Purchase Type against Payment Method, normalized by row (index) so each Purchase Type's payment methods sum to 100%.

What changed: Credit Card is the dominant payment method for both channels (62.02% Online, 58.25% Station), but Debit Card usage is notably higher at the Station (7.91%) than Online (3.48%), while Contactless usage is nearly identical across both channels (34.50% Online vs. 33.84% Station).

Purchase Type	Contactless %	Credit Card %	Debit Card %
Online	34.50	62.02	3.48
Station	33.84	58.25	7.91

Q16. How does the time of purchase influence the passenger's choice between Online and Station ticketing?

Why: Helps Sales/Marketing understand whether purchase channel preference shifts throughout the day (e.g., commuters buying at the station right before travel vs. planning trips online in advance).

How: Converted Time of Purchase to a datetime and extracted the hour, then plotted a count plot of Purchase Hour split by Purchase Type (Online vs. Station) as two side-by-side colored bars per hour.

What changed: Ticket purchase volume by hour and channel shows distinct patterns across the 24-hour day for Online versus Station purchases (see chart below for the hourly breakdown).

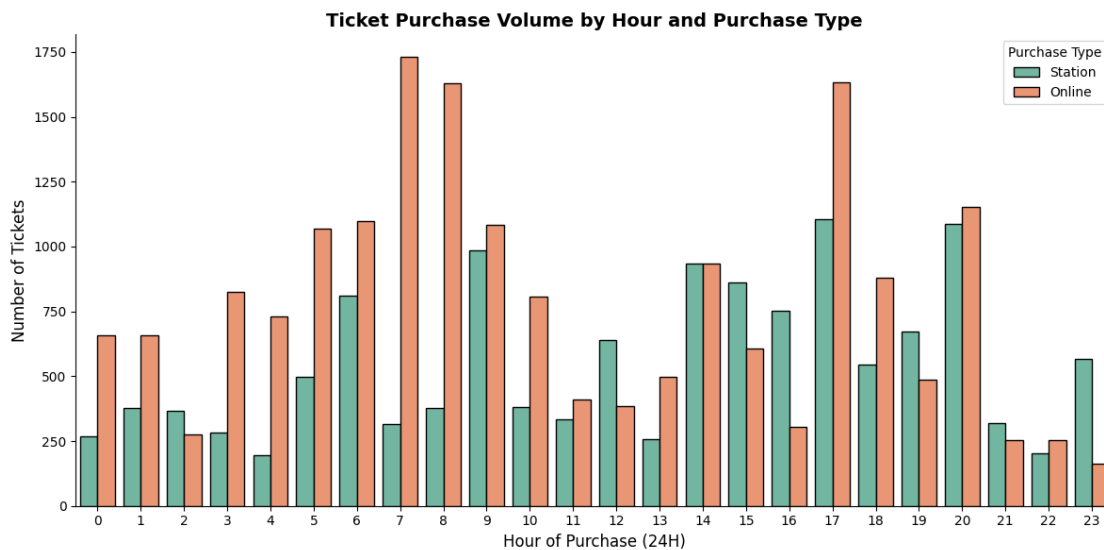


Figure: Ticket Purchase Volume by Hour and Purchase Type.

4. Multivariate Analysis

This layer examines interactions between three or more variables simultaneously, to uncover deeper, multidimensional patterns — such as how departure hour and ticket type together expose discount leakage, or how delay reason and route together drive the longest delays. The notebook defines 5 advanced business questions for this layer.

Q1. How does the average ticket price vary across the top 5 busiest routes when comparing different Ticket Types (Advance, Off-Peak, Anytime)?

Why: Combines the busiest-route finding from Univariate Q2 with the ticket-type discount concern from Problem Statement 02, to see whether the company's highest-demand routes are also where the Advance discount is eroding revenue the most.

How: Filtered to the 5 busiest routes by trip count, then plotted a grouped bar chart of average Price by Route, with bars split and colored by Ticket Type.

What changed: A large price disparity exists between Anytime and Advance tickets on these top 5 routes, with a substantial volume of tickets sold at the heavily discounted Advance rate even on the busiest, highest-demand corridors.

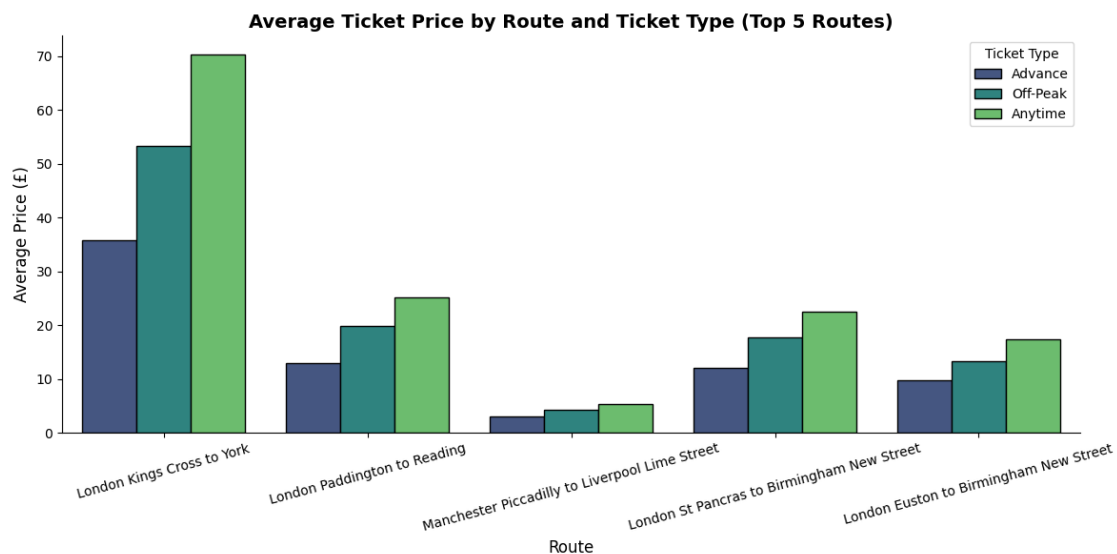


Figure: Average Ticket Price by Route and Ticket Type (Top 5 Routes).

Notebook insight — Observation: An analysis of the top 5 busiest routes reveals a massive price disparity between 'Anytime' and 'Advance' ticket categories. Despite being the highest-demand routes, a large volume of tickets is being sold at the heavily discounted 'Advance' rate.

Notebook insight — Business Impact ("So What?"): Since these specific routes experience consistently high passenger volume (inelastic demand), offering steep discounts represents a significant case of Revenue Leakage. The company is essentially sacrificing profit margins on passengers who are highly likely to travel regardless of the discount.

Notebook insight — Recommendation: Optimize the current Yield Management strategy by either reducing the allocation limit or decreasing the discount percentage for 'Advance' tickets exclusively on these top 5 routes. This minor adjustment will naturally shift passenger purchases toward 'Off-Peak' and 'Anytime' tiers, directly maximizing overall revenue without incurring any additional operational cost.

Q2. Which routes experience the highest average delay minutes during specific times of the day?

Why: Combines route-level and time-of-day delay patterns (previously examined separately in Bivariate Q6-Q8) into a single view, to identify specific route-and-hour combinations that Operations should prioritize.

How: Filtered to rows with Delay in Minutes greater than 0, extracted Departure Hour, restricted to the top 7 most frequently delayed routes, built a pivot table of average delay minutes by Route (rows) and Departure Hour (columns), and rendered it as a heatmap.

What changed: The heatmap reveals which specific route-and-hour combinations produce the longest average delays, going beyond the route-only (Q6) and hour-only (Q8) views examined earlier.

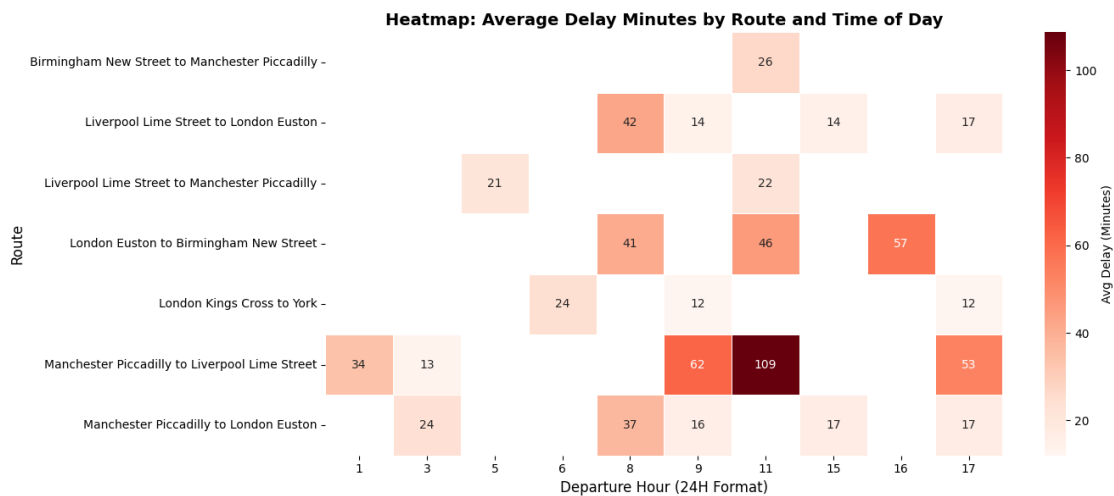


Figure: Heatmap of Average Delay Minutes by Route and Time of Day (Top 7 most-delayed routes).

Q3. Does the ticket class (First vs. Standard) influence the likelihood of a refund request across different delay durations?

Why: Extends Bivariate Q12 by adding Ticket Class as a third dimension, testing whether First Class passengers (who may have different expectations or compensation policies) behave differently from Standard passengers at the same delay lengths.

How: Filtered to rows with Delay in Minutes greater than 0, then plotted a box plot of Delay in Minutes by Refund Request, with a further split (hue) by Ticket Class.

What changed: The comparison surfaced a broader pattern about the refund/delay relationship itself, captured in the notebook's own insight below.

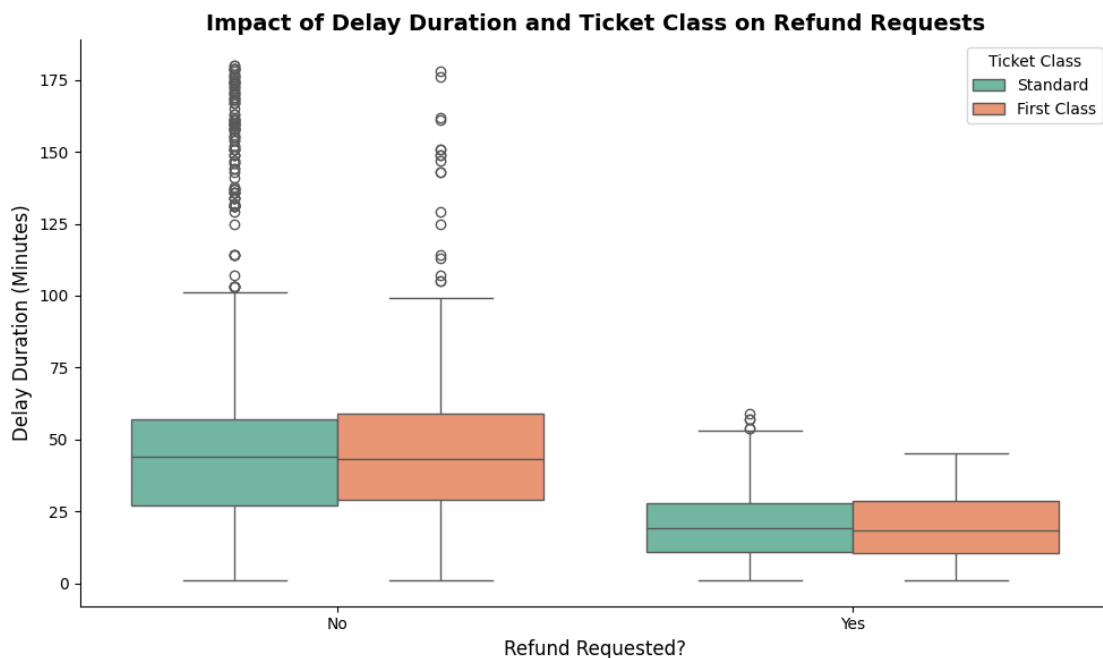


Figure: Impact of Delay Duration and Ticket Class on Refund Requests.

Notebook insight: "We can deduce that the systems of UK Train automatically refund after 60 min of delay. So passengers do not request refund[s]." — i.e., once a delay crosses roughly the 60-minute mark, the company's own systems appear to issue the refund automatically, which is why manually-requested refunds cluster below that delay length rather than increasing with delay duration as might be intuitively expected.

Q4. How does the average discount percentage vary across different departure hours when comparing Ticket Types?

Why: Directly follows up on the Off-Peak / peak-hour business rule check from the Data Cleaning phase, visually testing whether discount exposure spikes or dips around the defined peak windows (6-8 AM and 4-6 PM) for each ticket type.

How: Extracted Departure Hour, converted Total Discount % to a percentage scale, and plotted a multi-line chart of average discount by Departure Hour, one line per Ticket Type, with the two peak-hour windows (6-8 AM and 16-18) shaded for reference.

What changed: The shaded peak-hour bands make it possible to visually compare each ticket type's average discount inside versus outside peak windows, directly extending the earlier Data Cleaning check for Off-Peak rule violations into a full-day view across all ticket types.

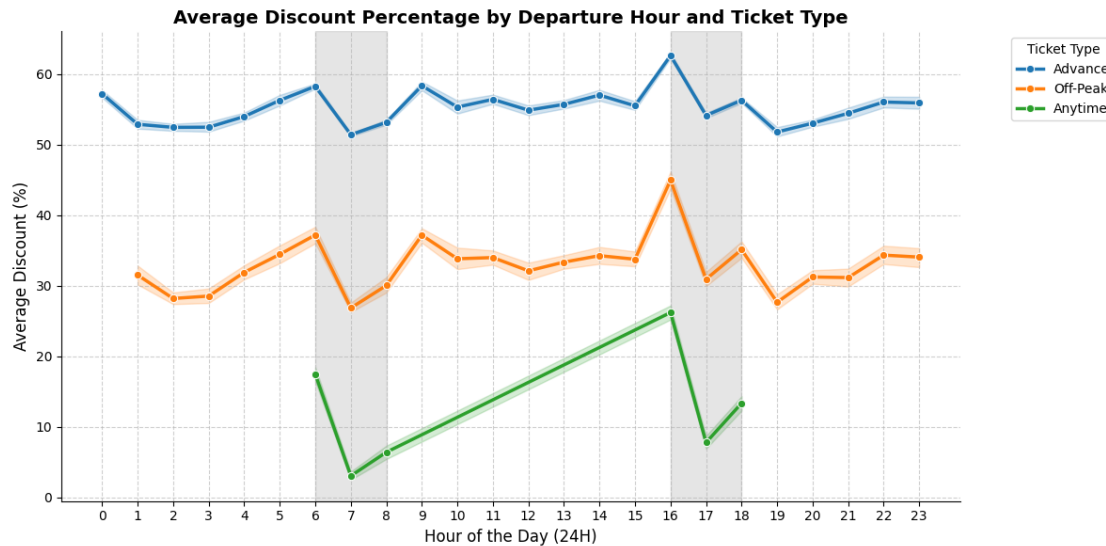


Figure: Average Discount Percentage by Departure Hour and Ticket Type (peak windows shaded).

Q5. Which combination of Route and Reason for Delay results in the longest average delay duration?

Why: The most granular operational question in the analysis — pinpointing exactly which route-and-cause combinations Operations Management should prioritize for intervention, rather than treating routes or delay reasons as independent factors.

How: Filtered to Delayed/Cancelled journeys, built a pivot table of average Delay in Minutes with Route as rows and Reason for Delay as columns, and rendered it as an annotated heatmap.

What changed: The heatmap identifies the specific route $\times$ delay-reason combinations with the highest average delay minutes, giving a far more targeted view than looking at routes (Bivariate Q6) or delay reasons (Bivariate Q7) in isolation.

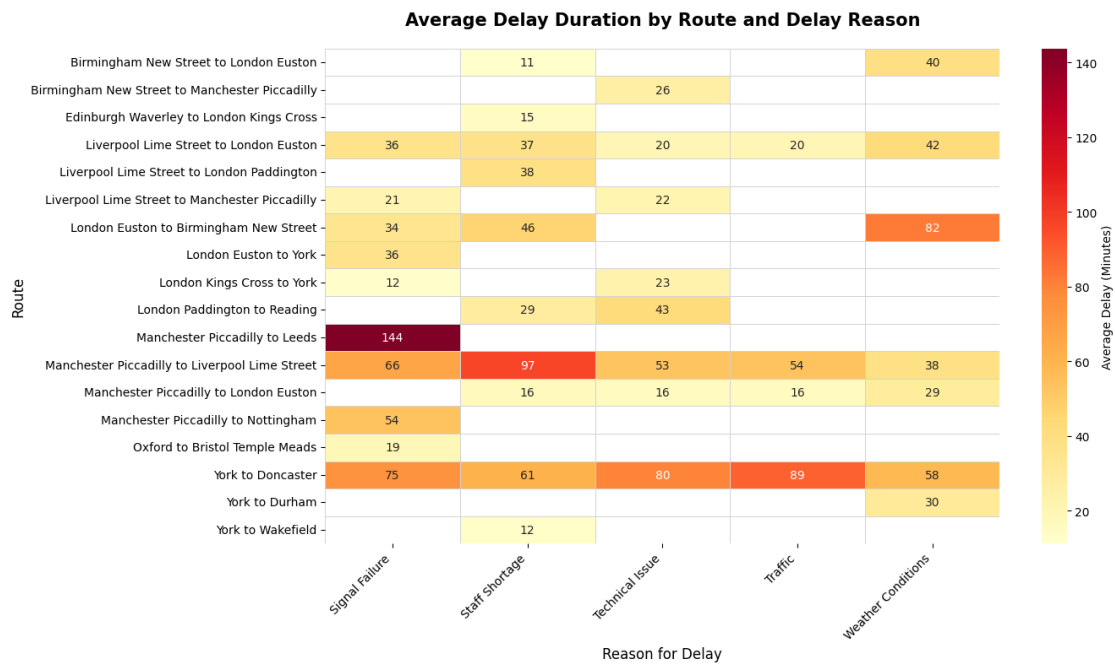


Figure: Average Delay Duration by Route and Delay Reason (heatmap).

5. Hypothesis Analysis

In this final section, the notebook transitions from exploratory analysis to formal hypothesis framing, stating that it is testing three core business hypotheses built on the correlations uncovered in the Bivariate and Multivariate sections. This is documented exactly as it exists in the notebook, including a gap: the notebook's markdown cells introduce three hypotheses under this heading, but only the first hypothesis has a corresponding analysis cell before the notebook ends — Hypotheses 2 and 3 are named in the intro text but have no executed code or stated conclusion in EDA.ipynb.

Hypothesis 1: The "Service Liability vs. Duration" Paradox

Null Hypothesis (H_0): The likelihood of a passenger requesting a refund is strictly dependent on the length of the delay (duration in minutes), regardless of the underlying reason.

Alternative Hypothesis (H_1): The likelihood of a refund request is driven by accountability (liability). Passengers exhibit zero tolerance for internal operational faults (Technical/Staff issues), triggering high refund rates even during shorter delay durations, compared to longer, uncontrollable disruptions (Weather).

Why: This hypothesis was formulated directly from the pattern observed in Bivariate Q7 and Q11 — where Technical Issue and Staff Shortage had shorter average delays than Signal Failure or Weather Conditions, yet Q10 showed Technical Issue driving the highest refund count and financial loss — suggesting refund behavior may track the reason for delay rather than (or in addition to) its raw duration.

How: The notebook re-runs the exact same analysis as Bivariate Q11: filtering to Delayed/Cancelled journeys, computing a binary refund flag, grouping by Reason for Delay, and taking the mean refund rate as a percentage, plotted as a bar chart.

Note for the team: No formal statistical test (e.g., a chi-square test of independence, logistic regression, or correlation coefficient) is computed in this cell — the notebook uses the same descriptive bar chart from Q11 as the evidence for this hypothesis, rather than a new inferential statistic. If a rigorous accept/reject conclusion on H_0 vs. H_1 is needed for the final report, a formal test (e.g., comparing refund rate across delay-reason groups while controlling for delay duration) would need to be added.

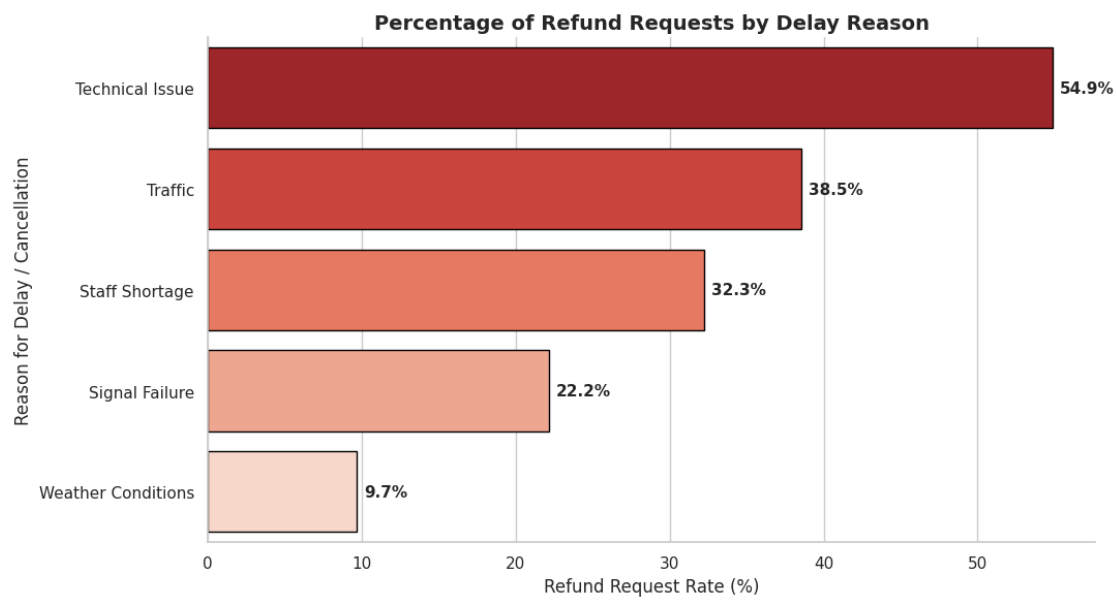


Figure: Percentage of Refund Requests by Delay Reason (repeated from Bivariate Q11, used here as supporting evidence for Hypothesis 1).

What we found: The chart shows refund request rates differ by Reason for Delay even though (per Bivariate Q7) these reasons do not carry the same average delay duration — Technical Issue and Staff Shortage, both classified as internal/controllable causes, produce comparatively high refund rates, which is consistent with the Alternative Hypothesis (H_1) that liability/accountability, not delay duration alone, shapes refund behavior. This is a descriptive/visual conclusion rather than a statistically tested one, per the note above.

Power BI Dashboard Documentation

Team Innovators - DEPI CAPSTONE PROJECT - UK Train Rides

Project Section 4 of 5: Power BI Dashboard

Source file: 1784073546312_Uk_Trains_Rides_Finale.pbix | Pages: Executive Summary, Revenue, Operational Overview, Operational Deepdive, Marketing

1. Project Overview

This Power BI report transforms the cleaned railway transaction dataset into an interactive analytical tool covering revenue, operations, and customer behaviour. The workflow that produced the report is:

Python (Data Cleaning & EDA) → GitHub Data Repository → Power BI Import → Star Schema Data Model → DAX Measure Tables → Five Interactive Dashboards

The report is organised into five pages — Executive Summary, Revenue, Operational Overview, Operational Deepdive, and Marketing — each built on the same underlying star-schema model so that filters and slicers behave consistently across the whole report.

2. Data Model

The model follows a Star Schema: a single fact table capturing every transaction, linked to a small set of dimension tables and three purpose-built measure tables.

Fact Table

f_transactions

Contains one row per ticket transaction, including keys, raw transactional fields, and pre-calculated financial columns:

- Transaction ID
- Route ID
- Attributes ID
- Date of Purchase
- Time of Purchase
- Date of Journey
- Departure Time
- Arrival Time
- Actual Arrival Time
- Price
- Price before discount
- Price After Refund
- Journey Duration
- Lead Time
- Delay in Minutes
- Railcard Discount
- Ticket Type Discount
- Total Discount %

Dimension Tables

d_routes

- Route ID
- Departure Station
- Arrival Destination
- Route

d_attributes

A merged lookup table holding every categorical/descriptive attribute of a journey:

- Attributes ID
- Purchase Type
- Payment Method
- Railcard
- Ticket Class
- Ticket Type
- Journey Status
- Refund Request
- Reason for Delay

Date & Time Dimensions

Table	Columns
d_date_purchase	Date
d_date_journey	Date, day number, Day of the month, Day of the week, Month Number
d_Time_Purchase	Time, Hour, Minute, SecondOfDay, Time Period
d_Time_Journey	Time, Hour, Minute, MinuteOfDay, Time half hours, Time Period

Relationships

All relationships run many-to-one from the fact table into each dimension, keeping the model a clean star schema (cardinality confirmed from the Power BI Model view diagram below):

From (fact table, many side)	To (dimension table, one side)
f_transactions[Route ID]	d_routes[Route ID]
f_transactions[Attributes ID]	d_attributes[Attributes ID]
f_transactions[Date of Purchase]	d_date_purchase[Date]
f_transactions[Date of Journey]	d_date_journey[Date]
f_transactions[Time of Purchase]	d_Time_Purchase[Time]
f_transactions[Departure Time / Arrival Time / Actual Arrival Time]	d_Time_Journey[Time]

Measure Tables

Three dedicated measure tables keep DAX business logic separate from the raw data tables.

_Revenue M

- Total Revenue
- Total Sales
- Total Sales after Discount
- Total refunds
- Loss from Discount
- loss from discount (%)
- Avg.Ticket Price
- Avg.Discount

_Operational M

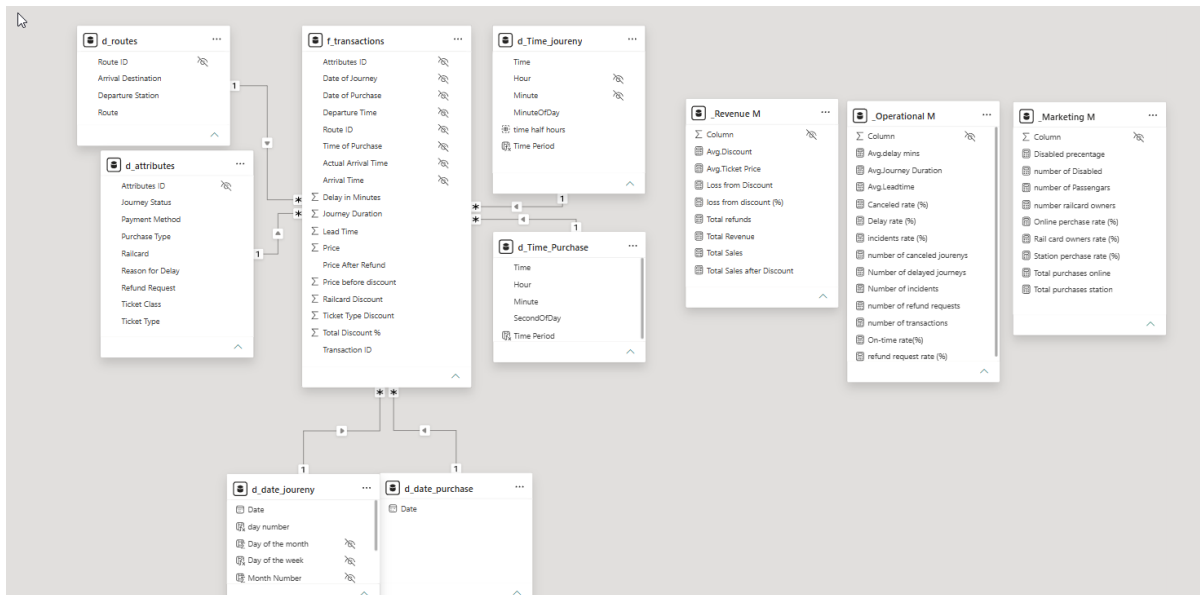
- number of transactions
- Number of incidents
- Number of delayed journeys
- number of canceled journeys
- number of refund requests
- Avg.delay mins
- Avg.Journey Duration
- Avg.Leadtime
- incidents rate (%)
- Delay rate (%)
- Canceled rate (%)
- refund request rate (%)
- On-time rate(%)

_Marketing M

- number of Passengars
- number railcard owners
- number of Disabled
- Disabled precentage
- Rail card owners rate (%)
- Total purchases online
- Total purchases station
- Online purchase rate (%)
- Station purchase rate (%)

Note: *A few measure names carry minor typos inherited from the model as built ("Passengars", "precentage", "Leadtime" without a space) — these are documented exactly as named in the .pbix so this report matches what's actually in the file; renaming them is a quick cleanup the team may want to do directly in Power BI.*

Data Model Diagram



Power BI Model view: fact table, dimensions, and measure tables.

3. Report Design

The report uses a consistent red-and-white executive theme across all five pages, with a navigation panel fixed to the left of every page for one-click movement between dashboards.

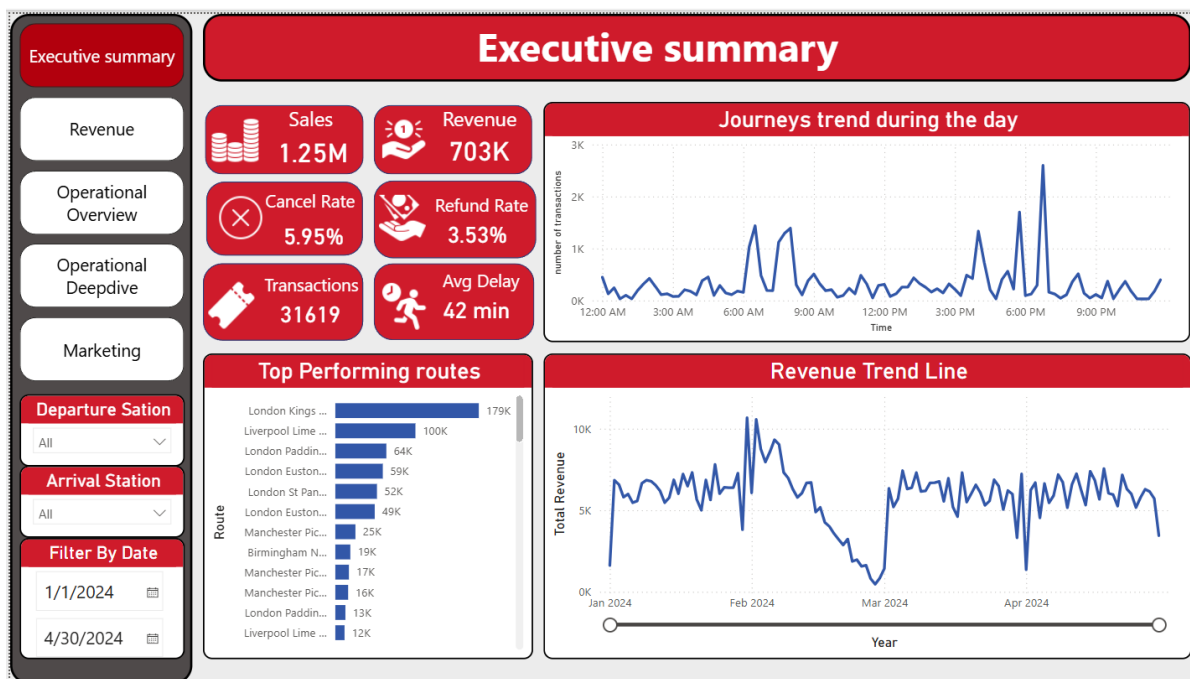
Common slicers on every page (synced across all 5 pages via matching Power BI sync groups):

- Departure Station
- Arrival Station
- Filter by Date (date range, default window 1/1/2024–4/30/2024)

The five report pages are:

- Executive Summary
- Revenue
- Operational Overview
- Operational Deepdive
- Marketing

4. Executive Summary



Executive Summary dashboard.

Purpose

Gives leadership a single-page, high-level view of sales, revenue, and operational health, alongside when passengers travel and which routes perform best.

Key Performance Indicators

1.25M	703K	5.95%	3.53%	31,619	42 min
Sales	Revenue	Cancel Rate	Refund Rate	Transactions	Avg Delay

Visuals

Journeys Trend During the Day: Line chart of transaction volume across the 24-hour clock, highlighting morning and evening peaks.

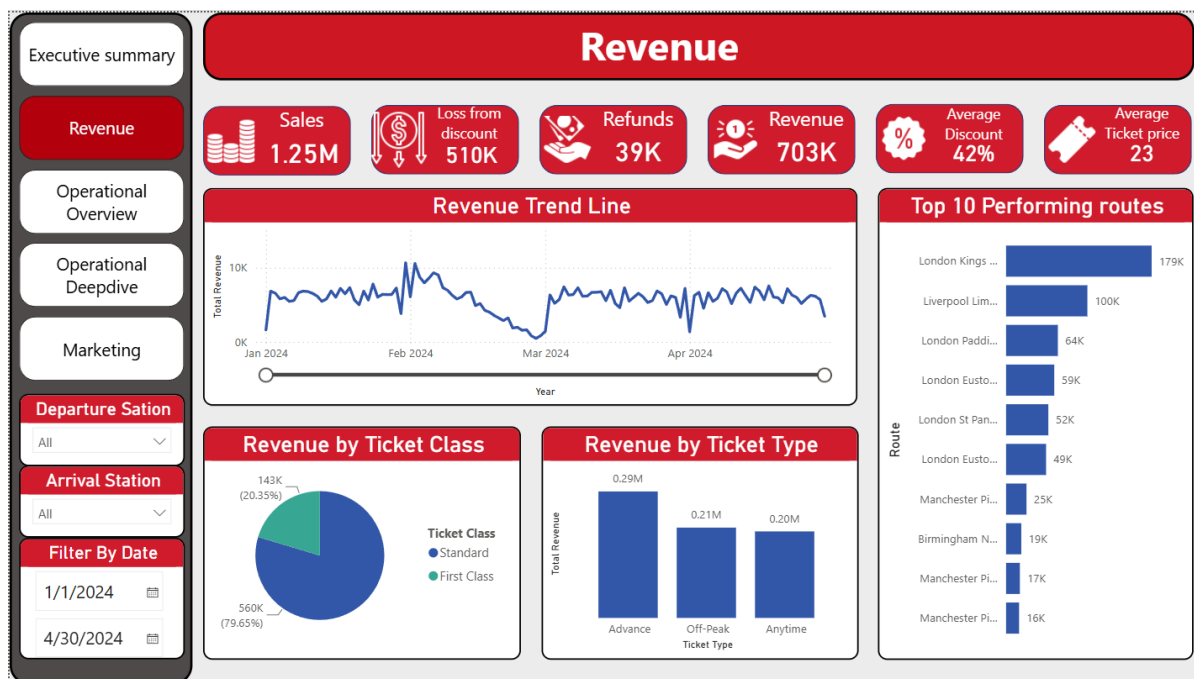
Top Performing Routes: Bar chart ranking routes by total revenue, led by London Kings Cross corridors.

Revenue Trend Line: Daily revenue from January to April 2024, with a slicer to adjust the date window.

Business Questions Answered

- How much revenue and how many sales has the business generated?
- How reliable is the network (cancel rate, refund rate, average delay)?
- Which routes generate the most revenue?
- When during the day do passengers travel?

5. Revenue



Revenue dashboard.

Purpose

Breaks down financial performance by ticket class, ticket type, and route to identify where revenue and margin are being won or lost.

Key Performance Indicators

1.25M	510K	39K	703K	42%	23
Sales	Loss from Discount	Refunds	Revenue	Avg Discount	Avg Ticket Price

Visuals

Revenue Trend Line: Daily revenue trend with a Jan–Apr 2024 date slider.

Revenue by Ticket Class: Pie chart: Standard Class contributes 79.65% of revenue versus 20.35% for First Class.

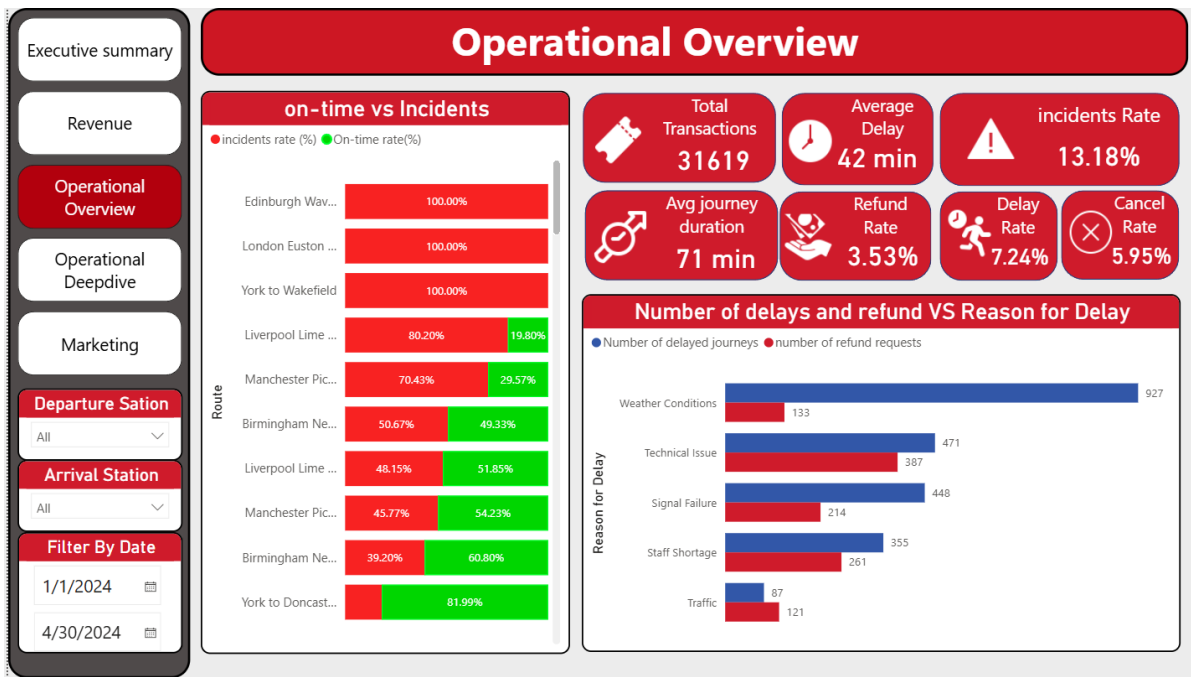
Revenue by Ticket Type: Column chart comparing Advance (0.29M), Off-Peak (0.21M), and Anytime (0.20M) revenue.

Top 10 Performing Routes: Horizontal bar ranking, led by London Kings Cross (179K) and Liverpool Lime Street (100K).

Business Questions Answered

- Which ticket class generates more revenue?
- Which ticket type sells best?
- How much revenue is lost to discounts?
- Which routes generate the highest income?

6. Operational Overview



Operational Overview dashboard.

Purpose

Monitors network reliability and pinpoints which routes and delay causes are driving incidents and refunds.

Key Performance Indicators

31,619	42 min	13.18%	71 min	3.53%	7.24%	5.95%
Transactions	Avg Delay	Incident Rate	Avg Journey Duration	Refund Rate	Delay Rate	Cancel Rate

Visuals

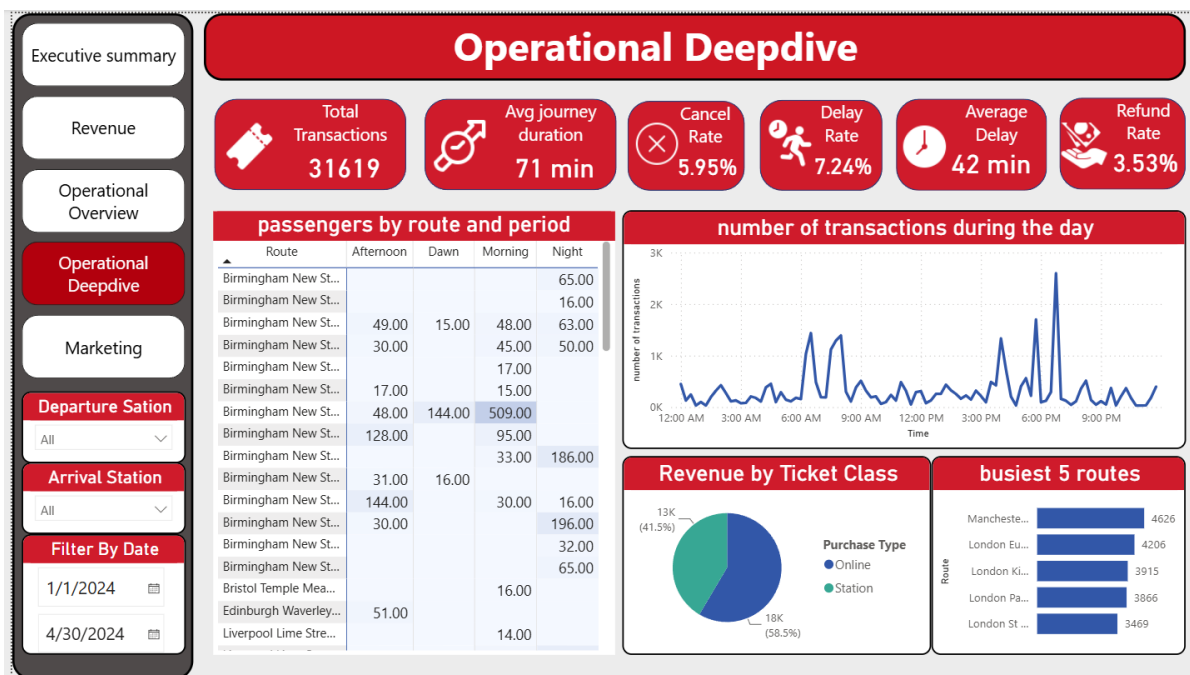
On-Time vs Incidents: Stacked bar comparing incident rate and on-time rate by route — several routes (Edinburgh Waverley, London Euston, York to Wakefield) show a 100% incident rate.

Number of Delays and Refunds vs. Reason for Delay: Dual bar chart comparing delayed-journey counts against refund-request counts for each delay cause; Weather Conditions produced the most delays (927) but relatively few refunds (133), while Technical Issue produced 471 delays and 387 refunds.

Business Questions Answered

- Which routes experience the highest delays or incidents?
- Why are passengers requesting refunds?
- How reliable is the railway network overall?

7. Operational Deepdive



Operational Deepdive dashboard.

Purpose

Provides a granular, route-and-time-period view of passenger volumes for detailed operational planning, supplementing the Operational Overview page.

Key Performance Indicators

31,619	71 min	5.95%	7.24%	42 min	3.53%
Transactions	Avg Journey Duration	Cancel Rate	Delay Rate	Avg Delay	Refund Rate

Visuals

Passengers by Route and Period: Pivot table breaking down passenger counts per route across Dawn, Morning, Afternoon, and Night periods.

Number of Transactions During the Day: Line chart of transaction volume across the 24-hour clock.

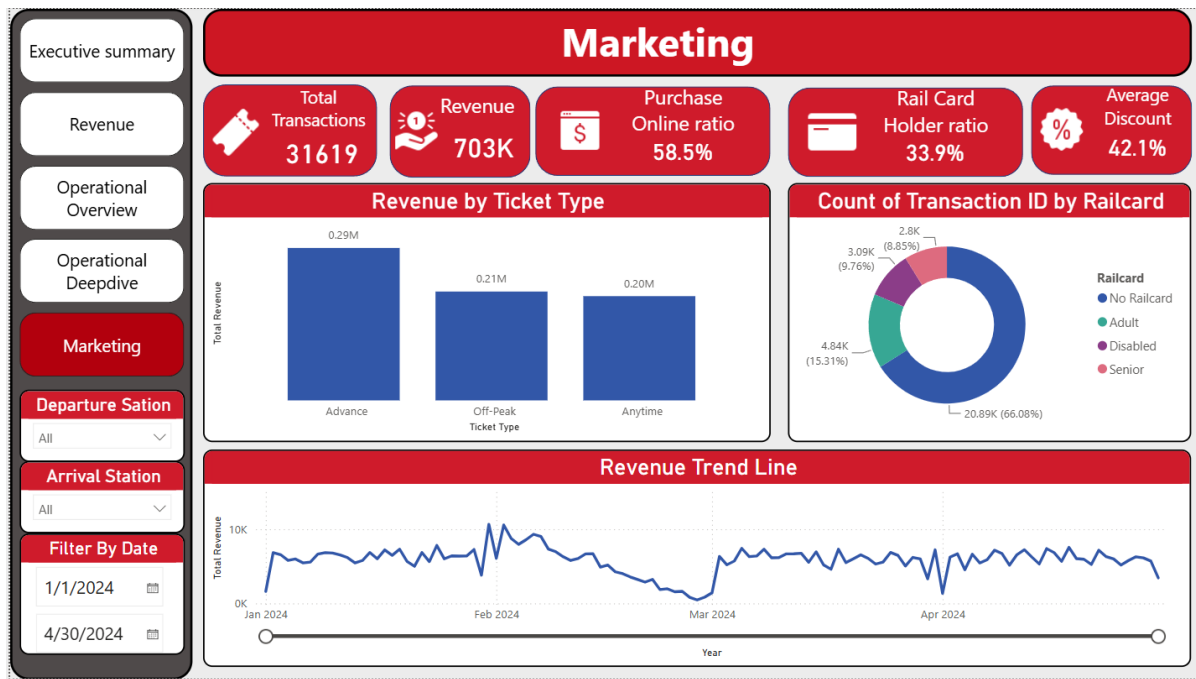
Revenue by Ticket Class (Purchase Type split): Pie chart split by Purchase Type — Station (58.5%) vs Online (41.5%).

Busiest 5 Routes: Bar chart led by Manchester Piccadilly corridors (4,626 transactions) and London Euston (4,206).

Business Questions Answered

- Which routes and time periods carry the most passengers?
- When during the day is transaction volume highest?
- Do passengers buy more online or at the station?
- Which are the busiest routes overall?

8. Marketing



Marketing dashboard.

Purpose

Analyzes purchasing behaviour — channel, railcard ownership, and discount usage — to inform marketing and pricing strategy.

Key Performance Indicators

31,619	703K	58.5%	33.9%	42.1%
Transactions	Revenue	Online Purchase Ratio	Railcard Holder Ratio	Avg Discount

Visuals

Revenue by Ticket Type: Column chart: Advance (0.29M) leads Off-Peak (0.21M) and Anytime (0.20M).

Count of Transaction ID by Railcard: Donut chart: No Railcard dominates at 66.08% (20.89K), followed by Adult (15.31%), Senior (9.76%), and Disabled (8.85%).

Revenue Trend Line: Daily revenue trend across Jan–Apr 2024.

Business Questions Answered

- Which ticket types are most popular?
- How many customers own a railcard, and of what type?
- What share of purchases happen online versus in station?
- How large is the average discount given to customers?

9. Filtering Behavior Notes

Confirmed from the report file: Inspection of the report's internal layout definition confirms the three slicers (Departure Station, Arrival Station, Filter by Date) share the same Power BI sync-group name on every one of the five pages. This means a filter selection made on one page (e.g., picking a single Departure Station on the Revenue page) carries over automatically when the user switches to any other page — the slicers are not independent per page.

Note: On several pages, KPI cards inherit a generic default title (e.g., several cards are internally still named "Total Revenue" even though they display different measures such as Refund Rate or Cancel Rate). This does not affect what end users see, since each card's visible label comes from a separate text/icon overlay rather than the visual's own title — but it's worth a mention here for whoever maintains the .pbix, since the internal titles no longer describe their measures.

Forecasting: Railway Journey Predictor

Team Innovators - DEPI CAPSTONE PROJECT - UK Train Rides

Project Section 5 of 5: Forecasting

1. Introduction

The Forecasting phase builds on the cleaned railway dataset and the insights from the Exploratory Data Analysis to deliver a predictive system: the Railway Journey Predictor. Given the details of a planned journey, the system estimates the ticket price, predicts whether the journey will run on time or be disrupted, and — if disruption is predicted — identifies the most likely cause.

The system is delivered as an interactive, terminal-based application built with the rich library, wrapping three CatBoost machine learning models behind a guided input flow and a readable results report.

2. Objectives

The forecasting system aims to:

- Predict the expected ticket price for a specified journey.
- Predict whether a journey is likely to run on time or be disrupted.
- Identify the most probable delay reason when disruption is predicted.
- Combine these predictions into a single Journey Intelligence score that supports a booking decision.
- Present results through an interactive, user-friendly command-line experience.

3. System Workflow

The end-to-end forecasting workflow follows these stages:

Cleaned Dataset (GitHub) → Feature Engineering → Chronological 80/20 Train-Test Split → Model Training (Price · Status · Delay Reason) → Model Caching (pickle) → Interactive Journey Input → Model Inference → Journey Intelligence Scoring & Result Display

4. Data Source

The system reuses the same cleaned dataset produced during the Data Cleaning phase, hosted on GitHub. If the CSV is not already present locally, it is downloaded automatically before training.

Trained models are cached to disk (`railway_models_v4.pkl`) alongside a version tag; the system only retrains when the cache is missing, unreadable, or its version no longer matches the current code, which keeps repeated runs fast.

5. Feature Engineering

Beyond the columns already produced during cleaning, the forecasting pipeline derives additional temporal and route features used exclusively by the models:

- **Journey_Month, Journey_DayOfWeek, Journey_DayOfMonth** — calendar components of the journey date.
- **Journey_IsWeekend** — flag for Saturday/Sunday journeys.
- **Departure_Hour** — hour extracted from the scheduled departure time.
- **Journey_IsPeakHour** — flag for departures in the 07:00–09:59 or 17:00–19:59 windows.

- **Journey_Season** — winter, spring, summer, or autumn, derived from the journey month.
- **Route** — departure station and arrival destination combined into one identifier.

Model Input Features

Type	Features
Categorical	Departure Station, Arrival Destination, Route, Railcard, Ticket Type, Ticket Class
Numerical	Lead Time, Journey_Month, Journey_DayOfWeek, Journey_DayOfMonth, Journey_IsWeekend, Journey_IsPeakHour, Journey_Season, Departure_Hour

6. Model Architecture

Three independent CatBoost models power the system, each targeting a different prediction.

Price Model

A CatBoostRegressor that predicts the ticket price for a journey based on its route, ticket attributes, and timing.

Hyperparameter	Value
Iterations	1,200
Learning Rate	0.04
Depth	6
L2 Leaf Reg	3
Min Data in Leaf	20

Journey Status Model

A CatBoostClassifier that predicts whether a journey will be "On Time" or "Disrupted", using balanced class weights to counter the natural imbalance between the two outcomes.

Hyperparameter	Value
Iterations	1,500
Learning Rate	0.03
Depth	8
Class Weights	Balanced
L2 Leaf Reg	5
Early Stopping Rounds	100

Delay Reason Model

A second CatBoostClassifier, trained only on journeys that were not on time, predicts the most likely cause of disruption (e.g. Signal Failure, Weather Conditions, Staff Shortage, Technical Issue). It shares the same hyperparameters as the Journey Status model and uses a stratified 80/20 split so every delay reason is represented in both the training and test sets.

Hyperparameter	Value
Iterations	1,500
Learning Rate	0.03
Depth	8
Class Weights	Balanced
L2 Leaf Reg	5
Early Stopping Rounds	100

7. Training Methodology

- The dataset is sorted chronologically by Date of Journey before splitting, so the model is tested on journeys that occurred after those it trained on — a more realistic validation than a random split.
- An 80/20 split is used for the Price and Journey Status models.
- The Delay Reason model uses a stratified 80/20 split restricted to disrupted journeys only, preserving the relative frequency of each delay reason.
- A fixed random seed (42) is used throughout for reproducibility.
- The Journey Status and Delay Reason models use early stopping after 100 rounds without improvement on the held-out evaluation set.

8. Model Evaluation

Each model is scored on the held-out test set, with results rated Excellent, Good, or Fair against fixed thresholds:

Model	Metric	Excellent	Good
Price	MAE (£)	≤ 5.00	≤ 15.00
Price	RMSE (£)	≤ 8.00	≤ 20.00
Price	R ²	≥ 0.85	≥ 0.70
Journey Status	Accuracy	$\geq 85\%$	$\geq 70\%$
Journey Status	F1 Score	≥ 0.85	≥ 0.70
Delay Reason	Accuracy	$\geq 75\%$	$\geq 60\%$
Delay Reason	F1 Score	≥ 0.75	≥ 0.60

9. Journey Intelligence Scoring

Beyond the three raw model outputs, every prediction is passed through a scoring routine that converts price, disruption risk, timing, and lead time into a single, easy-to-read assessment.

Journey Score (starts at 100 and is reduced by):

- Price above £120: –30 points; above £80: –20 points; above £50: –10 points.
- Predicted disruption: up to –40 points, scaled by the model's confidence in that prediction.
- Departure during peak hours (07:00–09:59 or 17:00–19:59): –10 points.
- Lead time under 3 days: –15 points; lead time under 7 days: –8 points.

The final Journey Score is capped between 0 and 100.

Derived Scores

Score	Formula
Risk Score	100 – Journey Score
Value Score	100 – (Predicted Price × 0.8), capped 0–100

Rating Tiers

Journey Score	Rating	Booking Advice
≥ 85	Excellent	BOOK NOW (≥ 80)
≥ 70	Good	CONSIDER (≥ 60)
≥ 50	Average	LOOK FOR ALTERNATIVES (< 60)
< 50	Poor	LOOK FOR ALTERNATIVES (< 60)

Each rating is paired with a short plain-English recommendation, e.g. "Excellent journey. Low disruption risk and good overall value." for top-tier scores, down to "High risk or poor value." for the lowest tier.

10. Interactive Application

The system runs as a guided command-line session rather than a one-shot script.

Input Collection

- Numbered menus for Departure Station, Arrival Destination, Ticket Class, Ticket Type, and Railcard.
- Free-entry prompts for Journey Date (YYYY-MM-DD), Departure Hour (0–23), and Lead Time (0–365 days).
- Departure and arrival stations are validated to ensure they are not the same.
- Routes not present in the training data are flagged as "unseen" and the result is marked as an extrapolation to be treated with caution.

Result Display

- A journey summary showing route, date, day of week, season, departure time, peak/off-peak status, lead time, class, type, and railcard.
- A Journey Intelligence Report with the Journey Score, Risk Score, Value Score, Rating, and Booking Advice.
- An AI Recommendation panel with a short plain-English summary.
- A predictions table showing estimated price, journey status with a confidence bar, and — when disrupted — the predicted delay reason with its own confidence bar.

- A final plain-English verdict panel summarising the price, status, and (if applicable) delay reason and confidence.

Session Summary

After one or more predictions, the user can request a session summary: a compact table listing every journey predicted so far (route, date, price, status, reason), together with aggregate statistics — total journeys, on-time vs. disrupted counts, average price, and price range.

11. Limitations

- Predictions for routes not seen during training are extrapolations and are explicitly flagged as lower-confidence.
- The models rely on historical route, timing, and ticket attributes only — no live weather, incident, or network-status feed is integrated.
- Hyperparameters are fixed by configuration rather than tuned through a search process, so further gains may be possible with systematic tuning.
- The Journey Intelligence score is a rule-based heuristic layered on top of the model outputs, not itself a trained or validated model.